☕ Spring Boot Microservices with REST API

Complete Notes: Beginner to Expert

> Covers every concept for learning, working, and acing interviews.
> Each section has theory, real code examples, and interview Q&A.

📋 Table of Contents

1. Spring Boot Fundamentals
2. REST API Design Principles
3. Building REST APIs with Spring Boot
4. Request Handling — Controllers & Mappings
5. Data Persistence — Spring Data JPA
6. Exception Handling
7. Validation
8. Microservices Architecture
9. Service Discovery — Eureka
10. API Gateway — Spring Cloud Gateway
11. Inter-Service Communication
12. Circuit Breaker — Resilience4j
13. Distributed Configuration — Spring Cloud Config
14. Security — Spring Security & JWT
15. Distributed Tracing — Sleuth & Zipkin
16. Message-Driven Microservices — Kafka & RabbitMQ
17. Testing — Unit, Integration & Contract
18. Docker & Kubernetes Integration
19. Monitoring — Actuator, Prometheus & Grafana
20. Best Practices & Patterns
21. Top 80 Interview Questions & Answers
22. Quick Reference Cheat Sheet

---

1. Spring Boot Fundamentals

What is Spring Boot?

Spring Boot is an **opinionated framework** built on top of Spring Framework that simplifies the setup, configuration, and deployment of Spring applications. It follows the **"convention over configuration"** principle.

Spring vs Spring Boot

Feature	Spring Framework	Spring Boot
Configuration	Manual XML / Java config	Auto-configuration
Server	Deploy WAR to Tomcat	Embedded Tomcat/Jetty
Startup	Complex setup	`@SpringBootApplication` + main()
Dependencies	Manual management	Starter POMs
Production ready	Manual setup	Actuator built-in

Core Annotations

```java
@SpringBootApplication   // = @Configuration + @EnableAutoConfiguration + @Component
@Configuration           // Marks class as source of bean definitions
@Component               // Generic Spring-managed bean
@Service                 // Business layer bean (semantic alias of @Component)
@Repository              // Data layer bean — enables exception translation
@Controller              // Web layer bean — returns view names
@RestController          // = @Controller + @ResponseBody — returns JSON/XML
@Bean                    // Declares a bean in a @Configuration class
@Autowired               // Dependency injection (field/constructor/setter)
@Value("${prop.key}")    // Inject property value
@Profile("dev")          // Activate bean only for a specific profile
@Conditional             // Conditional bean creation
@Primary                 // Preferred bean when multiple exist
@Qualifier("beanName")   // Specify which bean to inject
```

Spring Boot Project Structure

```
src/
├── main/
|   ├── java/com/example/
|   |   ├── ExpenseApplication.java          ← @SpringBootApplication main class
|   |   ├── controller/
|   |   |   └── ExpenseController.java       ← REST controllers
|   |   ├── service/
|   |   |   ├── ExpenseService.java          ← Interface
|   |   |   └── ExpenseServiceImpl.java      ← Implementation
|   |   ├── repository/
|   |   |   └── ExpenseRepository.java       ← JPA repository
|   |   ├── model/
|   |   |   └── Expense.java                 ← JPA entity
|   |   ├── dto/
|   |   |   ├── ExpenseRequest.java          ← Request DTO
|   |   |   └── ExpenseResponse.java         ← Response DTO
|   |   ├── exception/
|   |   |   ├── ExpenseNotFoundException.java
|   |   |   └── GlobalExceptionHandler.java
|   |   ├── config/
|   |   |   └── SecurityConfig.java
|   |   └── mapper/
|   |       └── ExpenseMapper.java
|   └── resources/
|       ├── application.yml                  ← Main config
|       ├── application-dev.yml              ← Dev profile
|       ├── application-prod.yml             ← Prod profile
|       └── db/migration/                    ← Flyway migrations
|           └── V1__create_expense_table.sql
└── test/
    └── java/com/example/
        ├── controller/
        |   └── ExpenseControllerTest.java
        └── service/
            └── ExpenseServiceTest.java
```

application.yml — Complete Example

```yaml
```

```yaml
spring:
  application:
    name: expense-service

  # Database
  datasource:
    url: jdbc:postgresql://localhost:5432/expensedb
    username: ${DB_USERNAME:postgres}
    password: ${DB_PASSWORD:secret}
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
      connection-timeout: 30000

  # JPA
  jpa:
    hibernate:
      ddl-auto: validate            # none | validate | update | create | create-dro
    show-sql: false
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
        format_sql: true

  # Profiles
  profiles:
    active: ${SPRING_PROFILE:dev}

# Server
server:
  port: 8081
  servlet:
    context-path: /api

# Logging
logging:
  level:
    com.example: DEBUG
    org.springframework.web: INFO
    org.hibernate.SQL: DEBUG

# Custom properties
app:
```

```yaml
  jwt:
    secret: ${JWT_SECRET:my-secret-key}
    expiration: 86400000           # 24 hours in ms
  pagination:
    default-page-size: 20
    max-page-size: 100
```

Dependency Injection Types

```java
// ✅ Constructor Injection (PREFERRED — immutable, testable)
@Service
public class ExpenseServiceImpl implements ExpenseService {

    private final ExpenseRepository expenseRepository;
    private final NotificationService notificationService;

    // @Autowired is optional if only one constructor
    public ExpenseServiceImpl(ExpenseRepository expenseRepository,
                              NotificationService notificationService) {
        this.expenseRepository = expenseRepository;
        this.notificationService = notificationService;
    }
}

// ❌ Field Injection (avoid — harder to test, hides dependencies)
@Service
public class ExpenseServiceImpl {
    @Autowired
    private ExpenseRepository expenseRepository;  // avoid this
}

// Setter Injection (for optional dependencies)
@Service
public class ExpenseServiceImpl {
    private NotificationService notificationService;

    @Autowired(required = false)
    public void setNotificationService(NotificationService svc) {
        this.notificationService = svc;
    }
}
```

Bean Lifecycle

```
1. Spring scans for @Component, @Service, @Repository, @Controller
2. Creates bean instances (via constructor)
3. Dependency injection (@Autowired)
4. @PostConstruct method called (initialisation)
5. Bean is ready — used by the application
6. @PreDestroy method called (before shutdown)
7. Bean destroyed

@PostConstruct
public void init() {
    log.info("ExpenseService initialised");
    // Load cache, validate config, etc.
}

@PreDestroy
public void cleanup() {
    log.info("ExpenseService shutting down");
    // Close connections, flush cache, etc.
}
```

---

2. REST API Design Principles

REST (Representational State Transfer) Principles

Principle	Description
Client-Server	Separation of UI and data concerns
Stateless	Each request contains all information needed — no session on server
Cacheable	Responses should indicate if they can be cached
Uniform Interface	Consistent resource naming and HTTP methods
Layered System	Client doesn't know if it talks to real server or proxy
Code on Demand	Optional: server can send executable code

HTTP Methods

Method	Action	Idempotent	Safe	Body
GET	Read	✅ Yes	✅ Yes	No
POST	Create	❌ No	❌ No	Yes
PUT	Replace (full update)	✅ Yes	❌ No	Yes
PATCH	Partial update	❌ No	❌ No	Yes
DELETE	Delete	✅ Yes	❌ No	No
HEAD	Like GET but no body	✅ Yes	✅ Yes	No
OPTIONS	Supported methods	✅ Yes	✅ Yes	No

HTTP Status Codes

```
2xx — Success
  200 OK              → Successful GET, PUT, PATCH
  201 Created         → Successful POST (include Location header)
  204 No Content      → Successful DELETE or PUT with no body

3xx — Redirection
  301 Moved Permanently  → Resource moved to new URL
  304 Not Modified       → Cached version is still valid

4xx — Client Errors
  400 Bad Request        → Invalid request syntax or validation failure
  401 Unauthorized       → Authentication required
  403 Forbidden          → Authenticated but not authorised
  404 Not Found          → Resource doesn't exist
  405 Method Not Allowed → HTTP method not supported
  409 Conflict           → Resource state conflict (duplicate email)
  422 Unprocessable      → Validation error (semantically wrong)
  429 Too Many Requests  → Rate limit exceeded

5xx — Server Errors
  500 Internal Server Error  → Unexpected server error
  502 Bad Gateway            → Upstream server error
  503 Service Unavailable    → Server overloaded or maintenance
  504 Gateway Timeout        → Upstream timeout
```

RESTful URL Design

```
Resource naming — ALWAYS use nouns, never verbs:

✅ Good:
GET    /expenses              → list all expenses
GET    /expenses/{id}         → get one expense
POST   /expenses              → create expense
PUT    /expenses/{id}         → full update
PATCH  /expenses/{id}         → partial update
DELETE /expenses/{id}         → delete expense

GET    /users/{userId}/expenses          → nested resource
GET    /expenses?category=FOOD&page=0   → filtering/pagination
GET    /expenses?sort=amount,desc        → sorting

❌ Bad (verbs in URL):
POST   /createExpense         → wrong
GET    /getExpenseById/1      → wrong
POST   /expenses/delete/1     → wrong
GET    /getAllExpenses         → wrong

✅ Versioning:
GET /api/v1/expenses          → URI versioning (most common)
GET /api/v2/expenses

Header versioning:
Accept: application/vnd.myapp.v1+json
```

Standard API Response Structure

```
json
```

```
{
  "success": true,
  "timestamp": "2025-01-15T10:30:00Z",
  "data": {
    "id": 1,
    "title": "Team Lunch",
    "amount": 48.50,
    "category": "FOOD"
  },
  "message": "Expense retrieved successfully"
}

// Error response:
{
  "success": false,
  "timestamp": "2025-01-15T10:30:00Z",
  "error": {
    "code": "EXPENSE_NOT_FOUND",
    "message": "Expense with id 999 not found",
    "details": []
  }
}

// Paginated list response:
{
  "success": true,
  "data": {
    "content": [...],
    "page": 0,
    "size": 20,
    "totalElements": 150,
    "totalPages": 8,
    "last": false
  }
}
```

---

3. Building REST APIs with Spring Boot

pom.xml — Essential Dependencies

```xml
```

```xml
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.4</version>
</parent>

<dependencies>
    <!-- Web / REST API -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Spring Data JPA -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- PostgreSQL Driver -->
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>
    </dependency>

    <!-- Validation -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <!-- Security -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <!-- Actuator (monitoring) -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
```

```
    <!-- Lombok (boilerplate reduction) -->
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>

    <!-- MapStruct (DTO mapping) -->
    <dependency>
        <groupId>org.mapstruct</groupId>
        <artifactId>mapstruct</artifactId>
        <version>1.5.5.Final</version>
    </dependency>

    <!-- Testing -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Entity

```
java
```

```java
@Entity
@Table(name = "expenses")
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Expense {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 150)
    private String title;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal amount;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Category category;

    @Column(nullable = false)
    private LocalDate date;

    @Column(length = 500)
    private String description;

    @CreationTimestamp
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @UpdateTimestamp
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    // Optimistic locking for concurrent updates
    @Version
    private Long version;
}
```

DTOs (Data Transfer Objects)

```java
```

```java
// Request DTO — what the client sends
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class ExpenseRequest {

    @NotBlank(message = "Title is required")
    @Size(max = 150, message = "Title must be under 150 characters")
    private String title;

    @NotNull(message = "Amount is required")
    @Positive(message = "Amount must be positive")
    @DecimalMax("999999.99")
    private BigDecimal amount;

    @NotNull(message = "Category is required")
    private Category category;

    @NotNull(message = "Date is required")
    @PastOrPresent(message = "Date cannot be in the future")
    private LocalDate date;

    @Size(max = 500)
    private String description;
}

// Response DTO — what the server returns
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class ExpenseResponse {
    private Long id;
    private String title;
    private BigDecimal amount;
    private Category category;
    private LocalDate date;
    private String description;
    private LocalDateTime createdAt;
}
```

MapStruct Mapper

```java
@Mapper(componentModel = "spring")
public interface ExpenseMapper {

    // Entity → Response DTO
    ExpenseResponse toResponse(Expense expense);

    // Request DTO → Entity
    @Mapping(target = "id", ignore = true)
    @Mapping(target = "createdAt", ignore = true)
    @Mapping(target = "updatedAt", ignore = true)
    @Mapping(target = "version", ignore = true)
    Expense toEntity(ExpenseRequest request);

    // List conversion
    List<ExpenseResponse> toResponseList(List<Expense> expenses);

    // Update entity from request (for PATCH operations)
    @BeanMapping(nullValuePropertyMappingStrategy = NullValuePropertyMappingStrategy
    void updateEntityFromRequest(ExpenseRequest request,
                                 @MappingTarget Expense expense);
}
```

Service Layer

```java
```

```java
public interface ExpenseService {
    ExpenseResponse create(ExpenseRequest request);
    ExpenseResponse findById(Long id);
    Page<ExpenseResponse> findAll(ExpenseFilterRequest filter, Pageable pageable);
    ExpenseResponse update(Long id, ExpenseRequest request);
    ExpenseResponse partialUpdate(Long id, ExpenseRequest request);
    void delete(Long id);
}

@Service
@RequiredArgsConstructor
@Slf4j
@Transactional(readOnly = true)          // default read-only for all methods
public class ExpenseServiceImpl implements ExpenseService {

    private final ExpenseRepository expenseRepository;
    private final ExpenseMapper expenseMapper;
    private final ApplicationEventPublisher eventPublisher;

    @Override
    @Transactional                        // override for write operations
    public ExpenseResponse create(ExpenseRequest request) {
        log.info("Creating expense: {}", request.getTitle());

        Expense expense = expenseMapper.toEntity(request);
        Expense saved = expenseRepository.save(expense);

        // Publish domain event
        eventPublisher.publishEvent(new ExpenseCreatedEvent(saved));

        return expenseMapper.toResponse(saved);
    }

    @Override
    public ExpenseResponse findById(Long id) {
        return expenseRepository.findById(id)
                .map(expenseMapper::toResponse)
                .orElseThrow(() -> new ExpenseNotFoundException(id));
    }

    @Override
    public Page<ExpenseResponse> findAll(ExpenseFilterRequest filter,
                                         Pageable pageable) {
```

```java
        Specification<Expense> spec = ExpenseSpecification.build(filter);
        return expenseRepository.findAll(spec, pageable)
                .map(expenseMapper::toResponse);
    }

    @Override
    @Transactional
    public ExpenseResponse update(Long id, ExpenseRequest request) {
        Expense expense = findExpenseOrThrow(id);
        expenseMapper.updateEntityFromRequest(request, expense);
        return expenseMapper.toResponse(expenseRepository.save(expense));
    }

    @Override
    @Transactional
    public void delete(Long id) {
        Expense expense = findExpenseOrThrow(id);
        expenseRepository.delete(expense);
        log.info("Deleted expense id={}", id);
    }

    private Expense findExpenseOrThrow(Long id) {
        return expenseRepository.findById(id)
                .orElseThrow(() -> new ExpenseNotFoundException(id));
    }
}
```

---

4. Request Handling — Controllers & Mappings

Complete REST Controller

```java
```

```java
@RestController
@RequestMapping("/api/v1/expenses")
@RequiredArgsConstructor
@Slf4j
@Tag(name = "Expenses", description = "Expense management API")   // OpenAPI
public class ExpenseController {

    private final ExpenseService expenseService;

    // GET /api/v1/expenses
    @GetMapping
    @Operation(summary = "List all expenses with pagination and filtering")
    public ResponseEntity<ApiResponse<Page<ExpenseResponse>>> getAll(
            @RequestParam(required = false) Category category,
            @RequestParam(required = false)
            @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate fromDate,
            @RequestParam(required = false)
            @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate toDate,
            @RequestParam(required = false) BigDecimal minAmount,
            @RequestParam(required = false) BigDecimal maxAmount,
            @PageableDefault(size = 20, sort = "date",
                             direction = Sort.Direction.DESC) Pageable pageable) {

        ExpenseFilterRequest filter = ExpenseFilterRequest.builder()
                .category(category)
                .fromDate(fromDate)
                .toDate(toDate)
                .minAmount(minAmount)
                .maxAmount(maxAmount)
                .build();

        Page<ExpenseResponse> expenses = expenseService.findAll(filter, pageable);
        return ResponseEntity.ok(ApiResponse.success(expenses));
    }

    // GET /api/v1/expenses/{id}
    @GetMapping("/{id}")
    public ResponseEntity<ApiResponse<ExpenseResponse>> getById(
            @PathVariable Long id) {
        ExpenseResponse expense = expenseService.findById(id);
        return ResponseEntity.ok(ApiResponse.success(expense));
    }
```

```java
    // POST /api/v1/expenses
    @PostMapping
    public ResponseEntity<ApiResponse<ExpenseResponse>> create(
            @Valid @RequestBody ExpenseRequest request) {
        ExpenseResponse created = expenseService.create(request);
        URI location = ServletUriComponentsBuilder
                .fromCurrentRequest()
                .path("/{id}")
                .buildAndExpand(created.getId())
                .toUri();
        return ResponseEntity
                .created(location)
                .body(ApiResponse.success(created, "Expense created successfully"));
    }

    // PUT /api/v1/expenses/{id}
    @PutMapping("/{id}")
    public ResponseEntity<ApiResponse<ExpenseResponse>> update(
            @PathVariable Long id,
            @Valid @RequestBody ExpenseRequest request) {
        ExpenseResponse updated = expenseService.update(id, request);
        return ResponseEntity.ok(ApiResponse.success(updated));
    }

    // PATCH /api/v1/expenses/{id}
    @PatchMapping("/{id}")
    public ResponseEntity<ApiResponse<ExpenseResponse>> partialUpdate(
            @PathVariable Long id,
            @RequestBody ExpenseRequest request) {        // no @Valid for PATCH
        ExpenseResponse updated = expenseService.partialUpdate(id, request);
        return ResponseEntity.ok(ApiResponse.success(updated));
    }

    // DELETE /api/v1/expenses/{id}
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        expenseService.delete(id);
        return ResponseEntity.noContent().build();        // 204 No Content
    }

    // GET /api/v1/expenses/summary/monthly
    @GetMapping("/summary/monthly")
    public ResponseEntity<ApiResponse<List<MonthlySummaryResponse>>> getMonthlySumma
            @RequestParam int year) {
```

```
        List<MonthlySummaryResponse> summary = expenseService.getMonthlySummary(year
        return ResponseEntity.ok(ApiResponse.success(summary));
    }
}
```

Parameter Annotations

```java
@PathVariable         // URL path segment:  /expenses/{id}
@RequestParam         // Query parameter:   /expenses?category=FOOD
@RequestBody          // Request body (JSON/XML)
@RequestHeader        // HTTP header value
@CookieValue          // Cookie value
@MatrixVariable       // Matrix variable:   /expenses/filter;cat=FOOD
@ModelAttribute       // Form data binding

// Examples:
public ResponseEntity<?> example(
    @PathVariable("id") Long expenseId,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(required = false) String search,
    @RequestHeader("Authorization") String authHeader,
    @RequestBody @Valid ExpenseRequest body
) {}
```

ApiResponse Wrapper

```java
```

```java
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class ApiResponse<T> {

    private boolean success;
    private String message;
    private T data;
    private LocalDateTime timestamp;
    private ErrorDetails error;

    public static <T> ApiResponse<T> success(T data) {
        return ApiResponse.<T>builder()
                .success(true)
                .data(data)
                .timestamp(LocalDateTime.now())
                .build();
    }

    public static <T> ApiResponse<T> success(T data, String message) {
        return ApiResponse.<T>builder()
                .success(true)
                .data(data)
                .message(message)
                .timestamp(LocalDateTime.now())
                .build();
    }

    public static <T> ApiResponse<T> error(String code, String message) {
        return ApiResponse.<T>builder()
                .success(false)
                .error(new ErrorDetails(code, message))
                .timestamp(LocalDateTime.now())
                .build();
    }

    @Data
    @AllArgsConstructor
    public static class ErrorDetails {
        private String code;
        private String message;
        private List<String> details = new ArrayList<>();
```

```
        public ErrorDetails(String code, String message) {
            this.code = code;
            this.message = message;
        }
    }
}
```

Content Negotiation

```java
// Produce and consume specific media types
@PostMapping(
    path = "/expenses",
    consumes = MediaType.APPLICATION_JSON_VALUE,
    produces = MediaType.APPLICATION_JSON_VALUE
)

// Support multiple formats
@GetMapping(produces = {
    MediaType.APPLICATION_JSON_VALUE,
    MediaType.APPLICATION_XML_VALUE
})
```

---

5. Data Persistence — Spring Data JPA

Repository Layer

```java
```

```java
@Repository
public interface ExpenseRepository extends JpaRepository<Expense, Long>,
                                           JpaSpecificationExecutor<Expense> {

    // Method name query derivation
    List<Expense> findByCategory(Category category);
    List<Expense> findByCategoryAndDateBetween(Category category,
                                               LocalDate from, LocalDate to);
    Optional<Expense> findByTitleIgnoreCase(String title);
    List<Expense> findByAmountGreaterThan(BigDecimal amount);
    List<Expense> findTop10ByOrderByAmountDesc();

    // JPQL queries
    @Query("SELECT e FROM Expense e WHERE e.category = :category " +
           "AND e.date BETWEEN :from AND :to ORDER BY e.date DESC")
    List<Expense> findByCategoryAndDateRange(
            @Param("category") Category category,
            @Param("from") LocalDate from,
            @Param("to") LocalDate to);

    // Native SQL query
    @Query(value = "SELECT category, SUM(amount) as total, COUNT(*) as count " +
                   "FROM expenses WHERE YEAR(date) = :year " +
                   "GROUP BY category ORDER BY total DESC",
           nativeQuery = true)
    List<Object[]> getCategorySummary(@Param("year") int year);

    // Projection — return only specific fields
    @Query("SELECT e.category AS category, SUM(e.amount) AS total " +
           "FROM Expense e WHERE YEAR(e.date) = :year GROUP BY e.category")
    List<CategorySummaryProjection> getCategorySummaryProjection(@Param("year") int

    // Custom delete
    @Modifying
    @Transactional
    @Query("DELETE FROM Expense e WHERE e.category = :category")
    int deleteByCategory(@Param("category") Category category);

    // Check existence
    boolean existsByTitleAndDate(String title, LocalDate date);

    // Count
    long countByCategory(Category category);
```

```java
}

// Projection interface
public interface CategorySummaryProjection {
    Category getCategory();
    BigDecimal getTotal();
}
```

JPA Specification (Dynamic Queries)

```java
```

```java
public class ExpenseSpecification {

    public static Specification<Expense> build(ExpenseFilterRequest filter) {
        return Specification
                .where(hasCategory(filter.getCategory()))
                .and(dateFrom(filter.getFromDate()))
                .and(dateTo(filter.getToDate()))
                .and(amountMin(filter.getMinAmount()))
                .and(amountMax(filter.getMaxAmount()))
                .and(titleContains(filter.getSearchText()));
    }

    private static Specification<Expense> hasCategory(Category category) {
        return (root, query, cb) ->
                category == null ? null : cb.equal(root.get("category"), category);
    }

    private static Specification<Expense> dateFrom(LocalDate from) {
        return (root, query, cb) ->
                from == null ? null : cb.greaterThanOrEqualTo(root.get("date"), from
    }

    private static Specification<Expense> dateTo(LocalDate to) {
        return (root, query, cb) ->
                to == null ? null : cb.lessThanOrEqualTo(root.get("date"), to);
    }

    private static Specification<Expense> amountMin(BigDecimal min) {
        return (root, query, cb) ->
                min == null ? null : cb.greaterThanOrEqualTo(root.get("amount"), min
    }

    private static Specification<Expense> titleContains(String text) {
        return (root, query, cb) ->
                text == null ? null :
                cb.like(cb.lower(root.get("title")), "%" + text.toLowerCase() + "%")
    }
}
```

Entity Relationships

```java
```

```java
// One-to-Many
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL,
               fetch = FetchType.LAZY, orphanRemoval = true)
    private List<Expense> expenses = new ArrayList<>();
}

// Many-to-One
@Entity
public class Expense {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;
}

// Many-to-Many
@Entity
public class Expense {
    @ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
    @JoinTable(
        name = "expense_tags",
        joinColumns = @JoinColumn(name = "expense_id"),
        inverseJoinColumns = @JoinColumn(name = "tag_id")
    )
    private Set<Tag> tags = new HashSet<>();
}

// One-to-One
@Entity
public class User {
    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @JoinColumn(name = "profile_id")
    private UserProfile profile;
}
```

Transaction Management

```java
```

```java
@Service
public class ExpenseServiceImpl {

    // READ-ONLY transaction — optimises DB performance
    @Transactional(readOnly = true)
    public ExpenseResponse findById(Long id) { ... }

    // Write transaction
    @Transactional
    public ExpenseResponse create(ExpenseRequest request) { ... }

    // Custom propagation
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void auditLog(String action) { ... }

    // Rollback on specific exceptions
    @Transactional(rollbackFor = {PaymentException.class, EmailException.class})
    public void processExpense(ExpenseRequest request) { ... }

    // No rollback for checked exception
    @Transactional(noRollbackFor = NotFoundException.class)
    public void processWithoutRollback() { ... }
}
```

Flyway Database Migrations

```sql
```

```sql
-- src/main/resources/db/migration/V1__create_expense_table.sql
CREATE TABLE expenses (
    id          BIGSERIAL PRIMARY KEY,
    title       VARCHAR(150) NOT NULL,
    amount      DECIMAL(10, 2) NOT NULL,
    category    VARCHAR(50) NOT NULL,
    date        DATE NOT NULL,
    description VARCHAR(500),
    user_id     BIGINT REFERENCES users(id),
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    version     BIGINT DEFAULT 0
);

CREATE INDEX idx_expenses_user_id ON expenses(user_id);
CREATE INDEX idx_expenses_category ON expenses(category);
CREATE INDEX idx_expenses_date ON expenses(date);

-- V2__add_expense_tags.sql
CREATE TABLE tags (
    id   BIGSERIAL PRIMARY KEY,
    name VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE expense_tags (
    expense_id BIGINT REFERENCES expenses(id) ON DELETE CASCADE,
    tag_id     BIGINT REFERENCES tags(id),
    PRIMARY KEY (expense_id, tag_id)
);
```

---

6. Exception Handling

Custom Exceptions

```java
```

```java
// Base exception
public abstract class AppException extends RuntimeException {
    private final String errorCode;
    private final HttpStatus httpStatus;

    protected AppException(String message, String errorCode, HttpStatus httpStatus)
        super(message);
        this.errorCode = errorCode;
        this.httpStatus = httpStatus;
    }
}

// Specific exceptions
public class ExpenseNotFoundException extends AppException {
    public ExpenseNotFoundException(Long id) {
        super("Expense not found with id: " + id,
              "EXPENSE_NOT_FOUND",
              HttpStatus.NOT_FOUND);
    }
}

public class DuplicateExpenseException extends AppException {
    public DuplicateExpenseException(String title) {
        super("Expense already exists with title: " + title,
              "DUPLICATE_EXPENSE",
              HttpStatus.CONFLICT);
    }
}

public class InsufficientBudgetException extends AppException {
    public InsufficientBudgetException(BigDecimal available, BigDecimal requested) {
        super(String.format("Insufficient budget. Available: %.2f, Requested: %.2f",
                            available, requested),
              "INSUFFICIENT_BUDGET",
              HttpStatus.UNPROCESSABLE_ENTITY);
    }
}
```

Global Exception Handler

```java
```

```java
@RestControllerAdvice                    // = @ControllerAdvice + @ResponseBody
@Slf4j
public class GlobalExceptionHandler {

    // Handle custom application exceptions
    @ExceptionHandler(AppException.class)
    public ResponseEntity<ApiResponse<?>> handleAppException(AppException ex,
                                                             HttpServletRequest re
        log.warn("Application exception: {} at {}", ex.getMessage(), request.getRequ
        return ResponseEntity
                .status(ex.getHttpStatus())
                .body(ApiResponse.error(ex.getErrorCode(), ex.getMessage()));
    }

    // Handle validation errors from @Valid
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiResponse<?>> handleValidation(
            MethodArgumentNotValidException ex) {

        List<String> errors = ex.getBindingResult()
                .getFieldErrors()
                .stream()
                .map(fe -> fe.getField() + ": " + fe.getDefaultMessage())
                .collect(Collectors.toList());

        ApiResponse<?> response = ApiResponse.<Void>builder()
                .success(false)
                .error(new ApiResponse.ErrorDetails("VALIDATION_FAILED",
                        "Request validation failed", errors))
                .timestamp(LocalDateTime.now())
                .build();

        return ResponseEntity.badRequest().body(response);
    }

    // Handle constraint violations (@Validated on service layer)
    @ExceptionHandler(ConstraintViolationException.class)
    public ResponseEntity<ApiResponse<?>> handleConstraintViolation(
            ConstraintViolationException ex) {
        List<String> errors = ex.getConstraintViolations()
                .stream()
                .map(v -> v.getPropertyPath() + ": " + v.getMessage())
                .collect(Collectors.toList());
```

```java
        return ResponseEntity.badRequest()
                .body(ApiResponse.error("VALIDATION_FAILED", "Constraint violation")
    }

    // Handle type mismatch (wrong type in path variable)
    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    public ResponseEntity<ApiResponse<?>> handleTypeMismatch(
            MethodArgumentTypeMismatchException ex) {
        String msg = String.format("Parameter '%s' should be of type %s",
                ex.getName(), ex.getRequiredType().getSimpleName());
        return ResponseEntity.badRequest()
                .body(ApiResponse.error("TYPE_MISMATCH", msg));
    }

    // Handle optimistic locking failure
    @ExceptionHandler(OptimisticLockingFailureException.class)
    public ResponseEntity<ApiResponse<?>> handleOptimisticLock(
            OptimisticLockingFailureException ex) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
                .body(ApiResponse.error("CONCURRENT_UPDATE",
                        "Resource was modified by another request. Please retry."));
    }

    // Catch-all for unexpected errors
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiResponse<?>> handleAll(Exception ex,
                                                    HttpServletRequest request) {
        log.error("Unhandled exception at {}: {}", request.getRequestURI(), ex.getMe
        return ResponseEntity.internalServerError()
                .body(ApiResponse.error("INTERNAL_ERROR",
                        "An unexpected error occurred. Please try again."));
    }
}
```

---

7. Validation

Bean Validation Annotations

```java
```

```java
public class ExpenseRequest {

    @NotNull(message = "Title cannot be null")
    @NotBlank(message = "Title cannot be blank")
    @Size(min = 2, max = 150, message = "Title must be 2-150 characters")
    private String title;

    @NotNull
    @Positive(message = "Amount must be positive")
    @DecimalMin(value = "0.01", message = "Minimum amount is 0.01")
    @DecimalMax(value = "999999.99", message = "Maximum amount is 999,999.99")
    @Digits(integer = 6, fraction = 2)
    private BigDecimal amount;

    @NotNull
    @Email(message = "Must be a valid email address")
    private String userEmail;

    @NotBlank
    @Pattern(regexp = "^[A-Za-z0-9]+$",
             message = "Category must contain only letters and numbers")
    private String categoryCode;

    @NotNull
    @Past(message = "Date must be in the past")
    // or @PastOrPresent / @Future / @FutureOrPresent
    private LocalDate date;

    @Size(max = 500)
    private String description;

    @Valid                            // cascade validation into nested object
    private AddressRequest address;

    @NotEmpty(message = "At least one tag required")
    private List<@NotBlank String> tags;
}
```

Custom Validator

```java
```

```java
// 1. Create annotation
@Target({ElementType.FIELD, ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = ValidCategoryValidator.class)
public @interface ValidCategory {
    String message() default "Invalid expense category";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

// 2. Create validator
public class ValidCategoryValidator
        implements ConstraintValidator<ValidCategory, String> {

    private static final Set<String> VALID_CATEGORIES =
            Set.of("FOOD", "TRANSPORT", "UTILITIES", "ENTERTAINMENT",
                   "HEALTH", "SHOPPING", "EDUCATION", "OTHER");

    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {
        if (value == null) return true;  // let @NotNull handle null
        return VALID_CATEGORIES.contains(value.toUpperCase());
    }
}

// 3. Use it
public class ExpenseRequest {
    @ValidCategory
    private String category;
}
```

Cross-Field Validation

```java
```

```java
// 1. Class-level annotation
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = DateRangeValidator.class)
public @interface ValidDateRange {
    String message() default "fromDate must be before toDate";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

// 2. Class-level validator
public class DateRangeValidator
        implements ConstraintValidator<ValidDateRange, ExpenseFilterRequest> {

    @Override
    public boolean isValid(ExpenseFilterRequest filter,
                           ConstraintValidatorContext context) {
        if (filter.getFromDate() == null || filter.getToDate() == null) {
            return true;
        }
        return !filter.getFromDate().isAfter(filter.getToDate());
    }
}

// 3. Apply to class
@ValidDateRange
public class ExpenseFilterRequest {
    private LocalDate fromDate;
    private LocalDate toDate;
}
```

---

8. Microservices Architecture

What are Microservices?

Microservices is an architectural style that structures an application as a collection of **small, independently deployable services**, each running its own process and communicating through well-defined APIs.

Monolith vs Microservices

Aspect	Monolith	Microservices
Deployment	Deploy entire app	Deploy individual services
Scaling	Scale entire app	Scale individual services
Technology	One stack	Polyglot (different stacks)
Team size	One large team	Small autonomous teams
Failure	One failure can crash app	Failure isolated to one service
Complexity	Simple initially	Operationally complex
Data	Single shared DB	Each service owns its DB
Communication	In-process method calls	Network API calls

Microservices Design Principles

```
1. Single Responsibility    → each service does one thing well
2. Database per Service     → services don't share databases
3. Design for Failure       → assume services will fail
4. API First                → define API contract before implementation
5. Smart Endpoints          → business logic in services, not pipes
6. Decentralised Data       → each service manages its own data
7. Infrastructure Automation → CI/CD, containerisation
8. Evolutionary Design      → services can be replaced independently
```

Expense Tracker Microservices Breakdown

```
┌──────────────────────────────────────────────────────────────┐
│                        CLIENT (Angular)                      │
└──────────────────────────────┬───────────────────────────────┘
                               │
                      ┌────────▼────────┐
                      │    API Gateway  │  port: 8080
                      │  (Spring Cloud GW)  │
                      └────────┬────────┘
                               │
              ┌────────┬────────┬────────┬────────┐
              ▼        ▼        ▼        ▼
```

```
    ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐
    |   User   |   | Expense  |   |  Budget  |   |  Report  |
    | Service  |   | Service  |   | Service  |   | Service  |
    |  :8081   |   |  :8082   |   |  :8083   |   |  :8084   |
    └────┬─────┘   └────┬─────┘   └────┬─────┘   └────┬─────┘
         |              |              |              |
    ┌────▼──┐      ┌────▼──┐      ┌────▼──┐      ┌────▼──┐
    |UserDB |      |ExpDB  |      |BudgDB |      |RepDB  |
    └───────┘      └───────┘      └───────┘      └───────┘

                                ↕ Kafka (async events)
                      ┌──────────────────────────┐
                      |    Notification Service  |
                      |          :8085           |
                      └──────────────────────────┘


Supporting Infrastructure:
  Eureka Server   :8761   (Service Discovery)
  Config Server   :8888   (Centralised Configuration)
  Zipkin          :9411   (Distributed Tracing)
  Prometheus      :9090   (Metrics)
  Grafana         :3000   (Dashboards)
```

Service Communication Patterns

```
Synchronous (request-response):
  REST (HTTP)          → simple, standard, widely supported
  gRPC                 → high performance, binary, strong typing
  GraphQL              → flexible queries, reduces over-fetching

Asynchronous (event-driven):
  Message Queue        → Kafka, RabbitMQ, ActiveMQ
  Event Bus            → AWS EventBridge, Google Pub/Sub
  Event Streaming      → Kafka Streams, Apache Flink
```

---

9. Service Discovery — Eureka

Why Service Discovery?

In a dynamic microservices environment, service instances come and go. Hardcoding IP addresses is impractical. Service discovery allows services to find each other dynamically.

```
Without Discovery:      expense-service → http://192.168.1.10:8082  (hardcoded,
breaks if service moves)

With Discovery:
  Each service registers with Eureka on startup
  expense-service → Eureka: "where is budget-service?" → 192.168.1.12:8083
  (IP is looked up dynamically, handles multiple instances automatically)
```

Eureka Server

xml

```xml
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

java

```java
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

yaml

```yaml
# application.yml — Eureka Server
server:
  port: 8761

spring:
  application:
    name: eureka-server

eureka:
  instance:
    hostname: localhost
  client:
    register-with-eureka: false   # don't register itself
    fetch-registry: false         # don't fetch registry
  server:
    enable-self-preservation: false  # disable in dev
```

Eureka Client (each microservice)

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

```yaml
# application.yml — Client service
spring:
  application:
    name: expense-service    # service name used for discovery

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true
    instance-id: ${spring.application.name}:${server.port}
    lease-renewal-interval-in-seconds: 10
    lease-expiration-duration-in-seconds: 30
```

```java
// @EnableEurekaClient is auto-applied when dependency is present
@SpringBootApplication
public class ExpenseServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ExpenseServiceApplication.class, args);
    }
}
```

---

10. API Gateway — Spring Cloud Gateway

What is an API Gateway?

Single entry point for all client requests. It handles:

- Routing to microservices
- Authentication and authorisation
- Rate limiting
- Load balancing
- Request/response transformation
- Logging and monitoring

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

```yaml
```

```yaml
# application.yml — API Gateway
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true           # auto-discover services from Eureka
          lower-case-service-id: true

      routes:
        # User Service route
        - id: user-service
          uri: lb://user-service  # lb:// = load balanced via Eureka
          predicates:
            - Path=/api/v1/users/**
          filters:
            - RewritePath=/api/v1/users/(?<segment>.*), /api/v1/users/${segment}
            - AddRequestHeader=X-Gateway-Source, api-gateway
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20

        # Expense Service route
        - id: expense-service
          uri: lb://expense-service
          predicates:
            - Path=/api/v1/expenses/**
            - Method=GET,POST,PUT,PATCH,DELETE
          filters:
            - AddRequestHeader=X-User-Id, #{T(java.lang.System).getProperty('user.id

        # Auth route (no JWT filter)
        - id: auth-service
          uri: lb://user-service
          predicates:
            - Path=/api/v1/auth/**

      # Global filters
      default-filters:
        - DedupeResponseHeader=Access-Control-Allow-Origin
```

```yaml
        - name: CircuitBreaker
          args:
            name: globalCircuitBreaker
            fallbackUri: forward:/fallback

      globalcors:
        cors-configurations:
          '[/**]':
            allowedOrigins:
              - "http://localhost:4200"
            allowedMethods:
              - GET
              - POST
              - PUT
              - DELETE
              - OPTIONS
            allowedHeaders: "*"
            allowCredentials: true

server:
  port: 8080
```

Custom Gateway Filter

```java
```

```java
@Component
@Slf4j
public class JwtAuthenticationFilter implements GlobalFilter, Ordered {

    private static final List<String> PUBLIC_PATHS = List.of(
            "/api/v1/auth/login",
            "/api/v1/auth/register",
            "/actuator/health"
    );

    private final JwtService jwtService;

    @Override
    public Mono<Void> filter(ServerWebExchange exchange,
                             GatewayFilterChain chain) {
        String path = exchange.getRequest().getPath().value();

        // Skip auth for public paths
        if (PUBLIC_PATHS.stream().anyMatch(path::startsWith)) {
            return chain.filter(exchange);
        }

        // Extract and validate JWT
        String authHeader = exchange.getRequest()
                .getHeaders()
                .getFirst(HttpHeaders.AUTHORIZATION);

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        String token = authHeader.substring(7);
        try {
            String userId = jwtService.extractUserId(token);

            // Forward user ID to downstream services
            ServerHttpRequest modifiedRequest = exchange.getRequest()
                    .mutate()
                    .header("X-User-Id", userId)
                    .header("X-User-Role", jwtService.extractRole(token))
                    .build();
```

```
            return chain.filter(exchange.mutate().request(modifiedRequest).build());

        } catch (JwtException e) {
            log.warn("JWT validation failed: {}", e.getMessage());
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }
    }

    @Override
    public int getOrder() {
        return -1;  // run before other filters
    }
}
```

---

11. Inter-Service Communication

RestTemplate (Legacy — use for Spring Boot 2.x)

```java
```

```java
@Configuration
public class RestTemplateConfig {

    @Bean
    @LoadBalanced          // enables service name resolution via Eureka
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@Service
@RequiredArgsConstructor
public class ExpenseService {

    private final RestTemplate restTemplate;

    public UserResponse getUserById(Long userId) {
        // 'user-service' is resolved by Eureka load balancer
        return restTemplate.getForObject(
                "http://user-service/api/v1/users/" + userId,
                UserResponse.class);
    }

    public void createUserExpense(Long userId, ExpenseRequest request) {
        ResponseEntity<ApiResponse<ExpenseResponse>> response =
                restTemplate.exchange(
                        "http://expense-service/api/v1/expenses",
                        HttpMethod.POST,
                        new HttpEntity<>(request),
                        new ParameterizedTypeReference<>() {});
    }
}
```

WebClient (Modern — Reactive, preferred)

```java
```

```java
@Configuration
public class WebClientConfig {

    @Bean
    @LoadBalanced
    public WebClient.Builder webClientBuilder() {
        return WebClient.builder()
                .defaultHeader(HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON_
                .codecs(config -> config.defaultCodecs().maxInMemorySize(1024 * 1024
    }
}

@Service
@RequiredArgsConstructor
public class ExpenseService {

    private final WebClient.Builder webClientBuilder;

    // Non-blocking call
    public Mono<UserResponse> getUserByIdReactive(Long userId) {
        return webClientBuilder.build()
                .get()
                .uri("http://user-service/api/v1/users/{id}", userId)
                .retrieve()
                .onStatus(HttpStatus::is4xxClientError,
                          response -> Mono.error(new UserNotFoundException(userId))
                .onStatus(HttpStatus::is5xxServerError,
                          response -> Mono.error(new ServiceUnavailableException())
                .bodyToMono(UserResponse.class);
    }

    // Blocking (for non-reactive context)
    public UserResponse getUserById(Long userId) {
        return getUserByIdReactive(userId).block();
    }

    // Parallel calls
    public Mono<DashboardResponse> getDashboard(Long userId) {
        Mono<UserResponse> userMono = getUserByIdReactive(userId);
        Mono<List<ExpenseResponse>> expensesMono = getExpenses(userId);
        Mono<BudgetResponse> budgetMono = getBudget(userId);

        return Mono.zip(userMono, expensesMono, budgetMono)
```

```
                .map(tuple -> DashboardResponse.builder()
                        .user(tuple.getT1())
                        .expenses(tuple.getT2())
                        .budget(tuple.getT3())
                        .build());
    }
}
```

OpenFeign (Declarative REST Client — preferred for Spring Cloud)

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

```java
```

```java
// Enable Feign in main class
@SpringBootApplication
@EnableFeignClients
public class ExpenseServiceApplication { ... }

// Define Feign client — interface acts like a REST client
@FeignClient(name = "user-service",
             fallback = UserClientFallback.class)
public interface UserServiceClient {

    @GetMapping("/api/v1/users/{id}")
    UserResponse getUserById(@PathVariable("id") Long id);

    @PostMapping("/api/v1/users/{id}/notifications")
    void sendNotification(@PathVariable("id") Long userId,
                          @RequestBody NotificationRequest request);
}

// Fallback implementation (used when user-service is down)
@Component
public class UserClientFallback implements UserServiceClient {

    @Override
    public UserResponse getUserById(Long id) {
        // Return a default/cached response
        return UserResponse.defaultUser(id);
    }

    @Override
    public void sendNotification(Long userId, NotificationRequest request) {
        log.warn("Notification service unavailable — dropping notification for user
    }
}

// Use in service
@Service
@RequiredArgsConstructor
public class ExpenseServiceImpl {

    private final UserServiceClient userServiceClient;

    public ExpenseResponse createWithUser(Long userId, ExpenseRequest request) {
        UserResponse user = userServiceClient.getUserById(userId);
```

```
        // proceed with expense creation
    }
}
```

```yaml
# Feign configuration
feign:
  client:
    config:
      default:
        connectTimeout: 5000
        readTimeout: 10000
        loggerLevel: BASIC    # NONE | BASIC | HEADERS | FULL
  circuitbreaker:
    enabled: true
```

---

12. Circuit Breaker — Resilience4j

Why Circuit Breaker?

```
Problem:
  Expense Service → calls → Budget Service (which is slow/down)
  All threads in Expense Service blocked waiting → cascade failure

Circuit Breaker:
  CLOSED  → normal operation, calls go through
  OPEN    → budget service is failing, stop calling it immediately
            return fallback response (fast fail)
  HALF-OPEN → after timeout, allow a few calls to test recovery
```

```xml
```

```xml
<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

yaml

```yaml
resilience4j:
  circuitbreaker:
    instances:
      budget-service:
        sliding-window-size: 10          # evaluate last 10 calls
        failure-rate-threshold: 50       # open if 50%+ failures
        wait-duration-in-open-state: 10s # stay open for 10s
        permitted-number-of-calls-in-half-open-state: 3
        minimum-number-of-calls: 5       # need 5 calls to evaluate

  retry:
    instances:
      budget-service:
        max-attempts: 3
        wait-duration: 500ms
        retry-exceptions:
          - java.net.ConnectException
          - java.util.concurrent.TimeoutException

  ratelimiter:
    instances:
      budget-service:
        limit-for-period: 100           # max 100 calls per period
        limit-refresh-period: 1s
        timeout-duration: 0s

  bulkhead:
    instances:
      budget-service:
        max-concurrent-calls: 10        # max 10 concurrent calls
        max-wait-duration: 0ms
```

```java
@Service
@RequiredArgsConstructor
@Slf4j
public class BudgetServiceClient {

    private final WebClient.Builder webClientBuilder;

    // Circuit Breaker + Retry + Rate Limiter stacked
    @CircuitBreaker(name = "budget-service", fallbackMethod = "getBudgetFallback")
    @Retry(name = "budget-service")
    @RateLimiter(name = "budget-service")
    @Bulkhead(name = "budget-service", type = Bulkhead.Type.SEMAPHORE)
    public BudgetResponse getBudget(Long userId) {
        return webClientBuilder.build()
                .get()
                .uri("http://budget-service/api/v1/budgets/{userId}", userId)
                .retrieve()
                .bodyToMono(BudgetResponse.class)
                .block(Duration.ofSeconds(5));
    }

    // Fallback method — same return type, with exception parameter
    public BudgetResponse getBudgetFallback(Long userId, Exception ex) {
        log.warn("Budget service unavailable for user {}. Error: {}",
                userId, ex.getMessage());
        return BudgetResponse.builder()
                .userId(userId)
                .status("UNAVAILABLE")
                .message("Budget information temporarily unavailable")
                .build();
    }

    // TimeLimiter (timeout)
    @TimeLimiter(name = "budget-service")
    public CompletableFuture<BudgetResponse> getBudgetAsync(Long userId) {
        return CompletableFuture.supplyAsync(() -> getBudget(userId));
    }
}
```

13. Distributed Configuration — Spring Cloud Config

Why Centralised Config?

```
Problem: 50 microservices, each with its own application.yml
         Changing DB password requires updating 50 files + redeployment

Solution: Config Server — one place to manage all configurations
          Services fetch config on startup from Config Server
          Config changes can be applied without redeployment (@RefreshScope)
```

Config Server Setup

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-config-server</artifactId>
</dependency>
```

```java
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication { ... }
```

```yaml
```

```yaml
# Config Server application.yml
server:
  port: 8888
spring:
  application:
    name: config-server
  cloud:
    config:
      server:
        git:
          uri: https://github.com/myorg/config-repo
          default-label: main
          search-paths: '{application}'     # folder per service
          username: ${GIT_USERNAME}
          password: ${GIT_TOKEN}
          clone-on-start: true
```

Config Client Setup

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```

```yaml
# bootstrap.yml (loaded before application.yml)
spring:
  application:
    name: expense-service
  config:
    import: optional:configserver:http://config-server:8888
  cloud:
    config:
      fail-fast: true
      retry:
        max-attempts: 5
```

```java
```

```
// Reload config without restart
@RestController
@RefreshScope           // re-creates bean when /actuator/refresh is called
public class ExpenseController {

    @Value("${app.max-expense-amount:10000}")
    private BigDecimal maxExpenseAmount;
}

// Trigger refresh:
// POST http://expense-service:8082/actuator/refresh
// (In production, use Spring Cloud Bus to broadcast to all instances)
```

---

14. Security — Spring Security & JWT

JWT Authentication Flow

```
1. POST /api/v1/auth/login  { email, password }
2. Server validates credentials
3. Server creates JWT:
   Header:  { alg: HS256, typ: JWT }
   Payload: { sub: "user@email.com", userId: 1, roles: ["ROLE_USER"], exp: ... }
   Signature: HMACSHA256(base64(header) + "." + base64(payload), secret)
4. Server returns: { accessToken: "eyJ...", refreshToken: "eyJ..." }
5. Client includes in all requests:
   Authorization: Bearer eyJ...
6. Server validates JWT on each request (stateless — no session DB)
```

Dependencies

```xml
```

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.3</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.3</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.3</version>
</dependency>
```

JWT Service

```java
```

```java
@Service
@Slf4j
public class JwtService {

    @Value("${app.jwt.secret}")
    private String secret;

    @Value("${app.jwt.expiration:86400000}")
    private long expirationMs;

    @Value("${app.jwt.refresh-expiration:604800000}")
    private long refreshExpirationMs;

    private SecretKey getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(secret);
        return Keys.hmacShaKeyFor(keyBytes);
    }

    public String generateAccessToken(UserDetails userDetails) {
        return buildToken(userDetails, expirationMs,
                Map.of("type", "access",
                       "roles", userDetails.getAuthorities()));
    }

    public String generateRefreshToken(UserDetails userDetails) {
        return buildToken(userDetails, refreshExpirationMs,
                Map.of("type", "refresh"));
    }

    private String buildToken(UserDetails userDetails, long expiration,
                              Map<String, Object> extraClaims) {
        return Jwts.builder()
                .claims(extraClaims)
                .subject(userDetails.getUsername())
                .issuedAt(new Date())
                .expiration(new Date(System.currentTimeMillis() + expiration))
                .signWith(getSigningKey())
                .compact();
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }
```

```java
    public <T> T extractClaim(String token,
                              Function<Claims, T> claimsResolver) {
        Claims claims = Jwts.parser()
                .verifyWith(getSigningKey())
                .build()
                .parseSignedClaims(token)
                .getPayload();
        return claimsResolver.apply(claims);
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        try {
            String username = extractUsername(token);
            return username.equals(userDetails.getUsername())
                    && !isTokenExpired(token);
        } catch (JwtException e) {
            log.warn("Invalid JWT: {}", e.getMessage());
            return false;
        }
    }

    private boolean isTokenExpired(String token) {
        return extractClaim(token, Claims::getExpiration).before(new Date());
    }
}
```

Security Configuration

```java
```

```java
@Configuration
@EnableWebSecurity
@EnableMethodSecurity            // enables @PreAuthorize, @PostAuthorize
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;
    private final UserDetailsService userDetailsService;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
                .csrf(csrf -> csrf.disable())
                .cors(cors -> cors.configurationSource(corsConfigurationSource()))
                .sessionManagement(session ->
                        session.sessionCreationPolicy(SessionCreationPolicy.STATELES
                .authorizeHttpRequests(auth -> auth
                        // Public endpoints
                        .requestMatchers("/api/v1/auth/**").permitAll()
                        .requestMatchers("/actuator/health", "/actuator/info").permi
                        .requestMatchers(HttpMethod.GET, "/api/v1/expenses/public/**
                        // Admin only
                        .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
                        // Any authenticated user
                        .anyRequest().authenticated()
                )
                .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter
                .exceptionHandling(ex -> ex
                        .authenticationEntryPoint(unauthorizedEntryPoint())
                        .accessDeniedHandler(accessDeniedHandler())
                )
                .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);   // cost factor 12
    }

    @Bean
    public AuthenticationManager authenticationManager(
            AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
```

```
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(List.of("http://localhost:4200"));
        config.setAllowedMethods(List.of("GET","POST","PUT","PATCH","DELETE","OPTION
        config.setAllowedHeaders(List.of("*"));
        config.setAllowCredentials(true);
        config.setMaxAge(3600L);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource
        source.registerCorsConfiguration("/**", config);
        return source;
    }
}

// JWT Filter
@Component
@RequiredArgsConstructor
@Slf4j
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        String authHeader = request.getHeader(HttpHeaders.AUTHORIZATION);

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String jwt = authHeader.substring(7);
        String username = jwtService.extractUsername(jwt);

        if (username != null &&
            SecurityContextHolder.getContext().getAuthentication() == null) {
```

```
            UserDetails userDetails = userDetailsService.loadUserByUsername(username

            if (jwtService.isTokenValid(jwt, userDetails)) {
                UsernamePasswordAuthenticationToken authToken =
                        new UsernamePasswordAuthenticationToken(
                                userDetails, null, userDetails.getAuthorities());
                authToken.setDetails(new WebAuthenticationDetailsSource()
                        .buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }
        filterChain.doFilter(request, response);
    }
}
```

Method-Level Security

```java
@RestController
@RequestMapping("/api/v1/expenses")
public class ExpenseController {

    // Only ADMIN can access
    @PreAuthorize("hasRole('ADMIN')")
    @DeleteMapping("/bulk")
    public ResponseEntity<Void> bulkDelete() { ... }

    // Authenticated user can only access their own expenses
    @PreAuthorize("#userId == authentication.principal.id or hasRole('ADMIN')")
    @GetMapping("/users/{userId}")
    public ResponseEntity<?> getUserExpenses(@PathVariable Long userId) { ... }

    // Custom SpEL expression
    @PreAuthorize("@expenseSecurityService.canAccess(#id, authentication)")
    @GetMapping("/{id}")
    public ResponseEntity<?> getById(@PathVariable Long id) { ... }
}
```

---

15. Distributed Tracing — Sleuth & Zipkin

Why Distributed Tracing?

```
A single user request flows through multiple services:
  API Gateway → User Service → Expense Service → Budget Service → Notification
Service

Without tracing:
  A request is slow. Which service is causing it? 🤷

With tracing:
  Every request gets a TraceId. Each service call gets a SpanId.
  Zipkin collects all spans, assembles the trace, shows timing per service.
```

Setup

```xml
<!-- Spring Boot 3.x -->
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
<dependency>
    <groupId>io.opentelemetry</groupId>
    <artifactId>opentelemetry-exporter-zipkin</artifactId>
</dependency>
```

```yaml
management:
  tracing:
    sampling:
      probability: 1.0       # 1.0 = trace 100% of requests (use 0.1 in prod)
  zipkin:
    tracing:
      endpoint: http://zipkin:9411/api/v2/spans

logging:
  pattern:
    level: "%5p [${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```

```
Log output now includes traceId and spanId:
  INFO [expense-service,65f5a2b1c3d4e5f6,1a2b3c4d] Creating expense...
  INFO [budget-service,65f5a2b1c3d4e5f6,5e6f7g8h]  Checking budget...
  ↑ same traceId = same request, different spanId = different service
```

---

16. Message-Driven Microservices — Kafka & RabbitMQ

Apache Kafka

```xml
<dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
</dependency>
```

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all            # wait for all replicas to acknowledge
      retries: 3
    consumer:
      group-id: expense-service
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeseriali
      auto-offset-reset: earliest
      properties:
        spring.json.trusted.packages: "com.example.*"
```

```java
```

```java
// Producer — publish events
@Service
@RequiredArgsConstructor
@Slf4j
public class ExpenseEventPublisher {

    private final KafkaTemplate<String, Object> kafkaTemplate;

    public static final String EXPENSE_CREATED_TOPIC = "expense-created";
    public static final String EXPENSE_DELETED_TOPIC = "expense-deleted";

    public void publishExpenseCreated(ExpenseCreatedEvent event) {
        kafkaTemplate.send(EXPENSE_CREATED_TOPIC,
                           String.valueOf(event.getUserId()),
                           event)
                .whenComplete((result, ex) -> {
                    if (ex != null) {
                        log.error("Failed to publish expense created event: {}", ex.
                    } else {
                        log.info("Published expense created: partition={}, offset={}
                                result.getRecordMetadata().partition(),
                                result.getRecordMetadata().offset());
                    }
                });
    }
}

// Consumer — listen for events
@Service
@Slf4j
public class NotificationEventConsumer {

    @KafkaListener(
            topics = "expense-created",
            groupId = "notification-service",
            containerFactory = "kafkaListenerContainerFactory"
    )
    public void handleExpenseCreated(
            @Payload ExpenseCreatedEvent event,
            @Header(KafkaHeaders.RECEIVED_TOPIC) String topic,
            @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
            Acknowledgment acknowledgment) {
```

```java
        log.info("Received expense created event: userId={}, amount={}",
                event.getUserId(), event.getAmount());

        try {
            // Process the event
            sendEmailNotification(event);
            checkBudgetLimit(event);

            // Manually acknowledge (only after successful processing)
            acknowledgment.acknowledge();

        } catch (Exception e) {
            log.error("Failed to process expense created event: {}", e.getMessage())
            // Don't acknowledge — message will be redelivered
            throw e;
        }
    }
}
```

RabbitMQ

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

```yaml
```

```yaml
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: guest
    password: guest
    virtual-host: /
    listener:
      simple:
        acknowledge-mode: manual
        retry:
          enabled: true
          max-attempts: 3
```

```java
```

```java
@Configuration
public class RabbitMQConfig {

    public static final String EXPENSE_QUEUE = "expense.created";
    public static final String EXPENSE_EXCHANGE = "expense.exchange";
    public static final String ROUTING_KEY = "expense.created";
    public static final String DLQ = "expense.dead-letter";

    @Bean
    public Queue expenseQueue() {
        return QueueBuilder.durable(EXPENSE_QUEUE)
                .withArgument("x-dead-letter-exchange", "")
                .withArgument("x-dead-letter-routing-key", DLQ)
                .build();
    }

    @Bean
    public DirectExchange expenseExchange() {
        return new DirectExchange(EXPENSE_EXCHANGE);
    }

    @Bean
    public Binding binding(Queue expenseQueue, DirectExchange expenseExchange) {
        return BindingBuilder.bind(expenseQueue)
                .to(expenseExchange)
                .with(ROUTING_KEY);
    }

    @Bean
    public MessageConverter messageConverter() {
        return new Jackson2JsonMessageConverter();
    }
}

// Producer
@Service
@RequiredArgsConstructor
public class ExpenseEventPublisher {

    private final RabbitTemplate rabbitTemplate;

    public void publish(ExpenseCreatedEvent event) {
        rabbitTemplate.convertAndSend(
```

```
                RabbitMQConfig.EXPENSE_EXCHANGE,
                RabbitMQConfig.ROUTING_KEY,
                event);
    }
}

// Consumer
@Service
@Slf4j
public class NotificationConsumer {

    @RabbitListener(queues = RabbitMQConfig.EXPENSE_QUEUE)
    public void handleExpenseCreated(ExpenseCreatedEvent event,
                                     Channel channel,
                                     @Header(AmqpHeaders.DELIVERY_TAG) long tag)
            throws IOException {
        try {
            processNotification(event);
            channel.basicAck(tag, false);      // acknowledge success
        } catch (Exception e) {
            channel.basicNack(tag, false, true); // nack — requeue
            log.error("Failed to process: {}", e.getMessage());
        }
    }
}
```

Kafka vs RabbitMQ

	Kafka	RabbitMQ
Model	Log-based (pull)	Queue-based (push)
Throughput	Very high (millions/sec)	High (thousands/sec)
Message retention	Time-based (configurable)	Until consumed
Replay	✅ Yes (rewind offset)	❌ No
Message ordering	Per partition	Per queue
Use case	Event streaming, audit log	Task queues, RPC

17. Testing — Unit, Integration & Contract

Unit Testing with JUnit 5 & Mockito

```java
```

```java
@ExtendWith(MockitoExtension.class)
class ExpenseServiceTest {

    @Mock
    private ExpenseRepository expenseRepository;

    @Mock
    private ExpenseMapper expenseMapper;

    @InjectMocks
    private ExpenseServiceImpl expenseService;

    @Test
    @DisplayName("Should create expense successfully")
    void createExpense_success() {
        // Given (Arrange)
        ExpenseRequest request = ExpenseRequest.builder()
                .title("Team Lunch")
                .amount(new BigDecimal("48.50"))
                .category(Category.FOOD)
                .date(LocalDate.now())
                .build();

        Expense expense = Expense.builder()
                .id(1L)
                .title("Team Lunch")
                .amount(new BigDecimal("48.50"))
                .build();

        ExpenseResponse expectedResponse = ExpenseResponse.builder()
                .id(1L)
                .title("Team Lunch")
                .build();

        when(expenseMapper.toEntity(request)).thenReturn(expense);
        when(expenseRepository.save(expense)).thenReturn(expense);
        when(expenseMapper.toResponse(expense)).thenReturn(expectedResponse);

        // When (Act)
        ExpenseResponse result = expenseService.create(request);

        // Then (Assert)
        assertThat(result).isNotNull();
```

```java
        assertThat(result.getId()).isEqualTo(1L);
        assertThat(result.getTitle()).isEqualTo("Team Lunch");

        verify(expenseRepository, times(1)).save(expense);
        verify(expenseMapper, times(1)).toEntity(request);
    }

    @Test
    @DisplayName("Should throw ExpenseNotFoundException when expense not found")
    void findById_notFound_throwsException() {
        // Given
        when(expenseRepository.findById(999L)).thenReturn(Optional.empty());

        // When & Then
        assertThatThrownBy(() -> expenseService.findById(999L))
                .isInstanceOf(ExpenseNotFoundException.class)
                .hasMessageContaining("999");

        verify(expenseRepository, times(1)).findById(999L);
    }

    @Test
    @DisplayName("Should delete expense and publish event")
    void deleteExpense_success() {
        // Given
        Expense expense = Expense.builder().id(1L).title("Test").build();
        when(expenseRepository.findById(1L)).thenReturn(Optional.of(expense));
        doNothing().when(expenseRepository).delete(expense);

        // When
        expenseService.delete(1L);

        // Then
        verify(expenseRepository, times(1)).delete(expense);
    }
}
```

Controller Test with MockMvc

```java
```

```java
@WebMvcTest(ExpenseController.class)
@Import({SecurityConfig.class})
class ExpenseControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private ExpenseService expenseService;

    @Autowired
    private ObjectMapper objectMapper;

    @Test
    @WithMockUser(username = "test@test.com", roles = "USER")
    @DisplayName("GET /expenses should return paginated list")
    void getAll_returnsPagedExpenses() throws Exception {
        // Given
        List<ExpenseResponse> expenses = List.of(
                ExpenseResponse.builder().id(1L).title("Lunch").build());
        Page<ExpenseResponse> page = new PageImpl<>(expenses);
        when(expenseService.findAll(any(), any())).thenReturn(page);

        // When & Then
        mockMvc.perform(get("/api/v1/expenses")
                        .param("page", "0")
                        .param("size", "20")
                        .contentType(MediaType.APPLICATION_JSON))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.success").value(true))
                .andExpect(jsonPath("$.data.content[0].id").value(1))
                .andExpect(jsonPath("$.data.content[0].title").value("Lunch"))
                .andDo(print());
    }

    @Test
    @WithMockUser
    @DisplayName("POST /expenses should create expense and return 201")
    void create_validRequest_returns201() throws Exception {
        // Given
        ExpenseRequest request = ExpenseRequest.builder()
                .title("Team Lunch")
                .amount(new BigDecimal("48.50"))
```

```java
                .category(Category.FOOD)
                .date(LocalDate.now())
                .build();

        ExpenseResponse response = ExpenseResponse.builder()
                .id(1L).title("Team Lunch").build();
        when(expenseService.create(any())).thenReturn(response);

        // When & Then
        mockMvc.perform(post("/api/v1/expenses")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(objectMapper.writeValueAsString(request)))
                .andExpect(status().isCreated())
                .andExpect(header().exists("Location"))
                .andExpect(jsonPath("$.data.id").value(1));
    }

    @Test
    @WithMockUser
    @DisplayName("POST /expenses with invalid data should return 400")
    void create_invalidRequest_returns400() throws Exception {
        // Missing required fields
        String invalidJson = """
            { "title": "", "amount": -10 }
        """;

        mockMvc.perform(post("/api/v1/expenses")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(invalidJson))
                .andExpect(status().isBadRequest())
                .andExpect(jsonPath("$.success").value(false))
                .andExpect(jsonPath("$.error.code").value("VALIDATION_FAILED"));
    }
}
```

Integration Testing with @SpringBootTest

```java
```

```java
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@Testcontainers
@Transactional
class ExpenseIntegrationTest {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15"
            .withDatabaseName("testdb")
            .withUsername("test")
            .withPassword("test");

    @DynamicPropertySource
    static void properties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired
    private TestRestTemplate restTemplate;

    @Autowired
    private ExpenseRepository expenseRepository;

    @Test
    @DisplayName("Full expense lifecycle: create → get → update → delete")
    void fullExpenseLifecycle() {
        // CREATE
        ExpenseRequest createRequest = ExpenseRequest.builder()
                .title("Integration Test Expense")
                .amount(new BigDecimal("100.00"))
                .category(Category.OTHER)
                .date(LocalDate.now())
                .build();

        ResponseEntity<ApiResponse<ExpenseResponse>> createResponse =
                restTemplate.postForEntity(
                        "/api/v1/expenses",
                        createRequest,
                        new ParameterizedTypeReference<>() {});

        assertThat(createResponse.getStatusCode()).isEqualTo(HttpStatus.CREATED);
```

```java
        Long id = createResponse.getBody().getData().getId();

        // GET
        ResponseEntity<ApiResponse<ExpenseResponse>> getResponse =
                restTemplate.getForEntity(
                        "/api/v1/expenses/" + id,
                        new ParameterizedTypeReference<>() {});

        assertThat(getResponse.getStatusCode()).isEqualTo(HttpStatus.OK);
        assertThat(getResponse.getBody().getData().getTitle())
                .isEqualTo("Integration Test Expense");

        // DELETE
        restTemplate.delete("/api/v1/expenses/" + id);
        assertThat(expenseRepository.findById(id)).isEmpty();
    }
}
```

Repository Tests with @DataJpaTest

```java
```

```java
@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@Testcontainers
class ExpenseRepositoryTest {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15"

    @DynamicPropertySource
    static void properties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired
    private ExpenseRepository expenseRepository;

    @Test
    void findByCategory_returnsMatchingExpenses() {
        // Given
        Expense foodExpense = expenseRepository.save(
                Expense.builder().title("Lunch").amount(BigDecimal.TEN)
                        .category(Category.FOOD).date(LocalDate.now()).build());
        expenseRepository.save(
                Expense.builder().title("Bus").amount(BigDecimal.ONE)
                        .category(Category.TRANSPORT).date(LocalDate.now()).build())

        // When
        List<Expense> results = expenseRepository.findByCategory(Category.FOOD);

        // Then
        assertThat(results).hasSize(1);
        assertThat(results.get(0).getTitle()).isEqualTo("Lunch");
    }
}
```

18. Docker & Kubernetes Integration

Dockerfile for Spring Boot

```dockerfile
# Multi-stage build
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY mvnw .
COPY .mvn .mvn
RUN ./mvnw dependency:resolve -q

COPY src src
RUN ./mvnw clean package -DskipTests -q

# Runtime stage (smaller image)
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# Security: non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

# Copy jar from builder
COPY --from=builder /app/target/*.jar app.jar

# JVM tuning for containers
ENTRYPOINT ["java",
    "-XX:+UseContainerSupport",
    "-XX:MaxRAMPercentage=75.0",
    "-XX:+UseG1GC",
    "-Djava.security.egd=file:/dev/./urandom",
    "-jar", "app.jar"]

EXPOSE 8082
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s \
  CMD wget -q --spider http://localhost:8082/actuator/health || exit 1
```

docker-compose.yml (local development)

```yaml
```

```yaml
version: '3.8'
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: expensedb
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: secret
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092

  eureka-server:
    build: ./eureka-server
    ports:
      - "8761:8761"
    healthcheck:
      test: ["CMD", "wget", "-q", "--spider", "http://localhost:8761/actuator/health

  config-server:
    build: ./config-server
    ports:
      - "8888:8888"
    depends_on:
      eureka-server:
        condition: service_healthy

  api-gateway:
```

```yaml
    build: ./api-gateway
    ports:
      - "8080:8080"
    environment:
      EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://eureka-server:8761/eureka/
    depends_on:
      - eureka-server
      - config-server

  expense-service:
    build: ./expense-service
    ports:
      - "8082:8082"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/expensedb
      SPRING_DATASOURCE_PASSWORD: secret
      EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://eureka-server:8761/eureka/
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
    depends_on:
      postgres:
        condition: service_healthy
      kafka:
        condition: service_started

volumes:
  postgres-data:
```

Kubernetes Deployment for Microservice

yaml

```yaml
# expense-service deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: expense-service
  namespace: expense-tracker
spec:
  replicas: 3
  selector:
    matchLabels:
      app: expense-service
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    metadata:
      labels:
        app: expense-service
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "8082"
        prometheus.io/path: "/actuator/prometheus"
    spec:
      containers:
        - name: expense-service
          image: myregistry/expense-service:2.0.0
          ports:
            - containerPort: 8082
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: "kubernetes"
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: expense-db-secret
                  key: password
            - name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE
              value: http://eureka-server:8761/eureka/
          resources:
            requests:
              memory: "256Mi"
```

```yaml
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: 8082
            initialDelaySeconds: 30
            periodSeconds: 10
            failureThreshold: 3
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: 8082
            initialDelaySeconds: 60
            periodSeconds: 15
            failureThreshold: 3
---
apiVersion: v1
kind: Service
metadata:
  name: expense-service
  namespace: expense-tracker
spec:
  selector:
    app: expense-service
  ports:
    - port: 8082
      targetPort: 8082
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: expense-service-hpa
  namespace: expense-tracker
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: expense-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
```

```yaml
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

---

19. Monitoring — Actuator, Prometheus & Grafana

Spring Boot Actuator

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

```yaml
```

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus,loggers,env,beans
      base-path: /actuator
  endpoint:
    health:
      show-details: when-authorized
      probes:
        enabled: true         # /health/liveness and /health/readiness
    info:
      enabled: true
  info:
    git:
      mode: full
    build:
      enabled: true

info:
  app:
    name: "@project.name@"
    version: "@project.version@"
    description: "@project.description@"
```

Custom Metrics

```java
```

```java
@Service
@RequiredArgsConstructor
public class ExpenseServiceImpl {

    private final MeterRegistry meterRegistry;
    private final Counter expenseCreatedCounter;
    private final Timer expenseCreationTimer;

    @PostConstruct
    private void initMetrics() {
        // Counter — increment on each create
        Counter.builder("expenses.created.total")
                .description("Total number of expenses created")
                .tag("service", "expense-service")
                .register(meterRegistry);

        // Timer — measure create operation time
        Timer.builder("expenses.creation.time")
                .description("Time to create an expense")
                .register(meterRegistry);

        // Gauge — current value
        Gauge.builder("expenses.active.count",
                       expenseRepository, ExpenseRepository::count)
                .description("Number of active expenses")
                .register(meterRegistry);
    }

    public ExpenseResponse create(ExpenseRequest request) {
        return expenseCreationTimer.record(() -> {
            Expense expense = expenseMapper.toEntity(request);
            Expense saved = expenseRepository.save(expense);
            meterRegistry.counter("expenses.created.total",
                                  "category", request.getCategory().name())
                         .increment();
            return expenseMapper.toResponse(saved);
        });
    }
}
```

Custom Health Indicator

```java
@Component
public class DatabaseHealthIndicator implements HealthIndicator {

    private final DataSource dataSource;

    @Override
    public Health health() {
        try (Connection conn = dataSource.getConnection()) {
            if (conn.isValid(2)) {
                return Health.up()
                        .withDetail("database", "PostgreSQL")
                        .withDetail("url", conn.getMetaData().getURL())
                        .build();
            }
            return Health.down()
                    .withDetail("reason", "Connection invalid")
                    .build();
        } catch (SQLException e) {
            return Health.down()
                    .withException(e)
                    .withDetail("reason", e.getMessage())
                    .build();
        }
    }
}
```

---

20. Best Practices & Patterns

DTO Pattern — Never Expose Entities Directly

```java
```

```java
// ❌ Bad — exposing JPA entity directly
@GetMapping("/{id}")
public Expense getById(@PathVariable Long id) {
    return expenseRepository.findById(id).orElseThrow();
    // Exposes: internal fields, lazy loading issues, security risks
}

// ✅ Good — use DTOs
@GetMapping("/{id}")
public ResponseEntity<ApiResponse<ExpenseResponse>> getById(@PathVariable Long id) {
    return ResponseEntity.ok(ApiResponse.success(expenseService.findById(id)));
}
```

Pagination Best Practices

```java
// Always paginate list endpoints
@GetMapping
public ResponseEntity<ApiResponse<Page<ExpenseResponse>>> getAll(
        @PageableDefault(size = 20, sort = "createdAt",
                         direction = Sort.Direction.DESC) Pageable pageable) {

    // Validate max page size
    if (pageable.getPageSize() > 100) {
        throw new ValidationException("Maximum page size is 100");
    }

    return ResponseEntity.ok(ApiResponse.success(expenseService.findAll(pageable)));
}
```

Idempotency for POST Requests

```java
```

```java
@PostMapping
public ResponseEntity<?> create(
        @RequestHeader(value = "Idempotency-Key", required = false) String idempoten
        @Valid @RequestBody ExpenseRequest request) {

    if (idempotencyKey != null) {
        // Check if we already processed this request
        Optional<ExpenseResponse> cached = idempotencyStore.get(idempotencyKey);
        if (cached.isPresent()) {
            return ResponseEntity.ok(ApiResponse.success(cached.get(),
                    "Duplicate request — returning cached result"));
        }
    }

    ExpenseResponse created = expenseService.create(request);

    if (idempotencyKey != null) {
        idempotencyStore.put(idempotencyKey, created);
    }

    return ResponseEntity.created(buildLocation(created.getId()))
            .body(ApiResponse.success(created));
}
```

Saga Pattern for Distributed Transactions

```
Problem: Creating an order involves User Service, Inventory Service, Payment
Service.
         One fails after others succeed → inconsistent state.

Choreography-based Saga:
  1. OrderService creates order → publishes OrderCreatedEvent
  2. InventoryService reserves stock → publishes StockReservedEvent
  3. PaymentService charges → publishes PaymentSuccessEvent
  4. OrderService confirms order
  On any failure → publish compensating event (rollback)
    PaymentFailed → InventoryService releases stock → OrderService cancels order

Orchestration-based Saga:
  1. SagaOrchestrator calls each step
```

```
  2. On failure, calls compensating transaction
  3. Central place for saga state and compensation
```

API Versioning Strategies

```java
// 1. URI versioning (most visible, most common)
@RequestMapping("/api/v1/expenses")
@RequestMapping("/api/v2/expenses")

// 2. Header versioning
@GetMapping(headers = "API-Version=1")
@GetMapping(headers = "API-Version=2")

// 3. Media type versioning
@GetMapping(produces = "application/vnd.myapp.v1+json")
@GetMapping(produces = "application/vnd.myapp.v2+json")

// 4. Parameter versioning
@GetMapping(params = "version=1")
```

Outbox Pattern (Reliable Event Publishing)

```
Problem: Service saves to DB and publishes to Kafka.
         DB commit succeeds but Kafka publish fails → event lost.

Outbox Pattern:
  1. Save entity AND event to DB in the SAME transaction
     (to an 'outbox' table)
  2. A separate process reads the outbox table and publishes to Kafka
  3. Mark events as published
  4. Guaranteed delivery — DB transaction is the source of truth

@Transactional
public ExpenseResponse create(ExpenseRequest request) {
    Expense expense = expenseRepository.save(toEntity(request));
    // Write event to outbox table in same transaction
    outboxRepository.save(new OutboxEvent("ExpenseCreated", toJson(expense)));
    return toResponse(expense);
}
// Separate scheduled task polls outbox and publishes
```

21. Top 80 Interview Questions & Answers

Spring Boot Basics (Q1–Q20)

Q1. What is Spring Boot and what problems does it solve?

> Spring Boot is an opinionated extension of Spring Framework that provides auto-configuration, embedded servers, and starter POMs. It solves: complex XML configuration, manual server setup, dependency version conflicts, and boilerplate setup code. You write business logic instead of infrastructure code.

Q2. What is auto-configuration in Spring Boot?

> Spring Boot scans the classpath and automatically configures beans based on what jars are present. Example: if `spring-boot-starter-web` is on the classpath, Spring Boot auto-configures `DispatcherServlet`, `Jackson`, `Tomcat`, etc. Controlled by `@EnableAutoConfiguration` in `@SpringBootApplication`. You can exclude auto-configurations via `@SpringBootApplication(exclude=DataSourceAutoConfiguration.class)`.

Q3. Explain @SpringBootApplication.

> It combines three annotations: `@Configuration` (marks class as bean configuration), `@EnableAutoConfiguration` (enables auto-config), and `@ComponentScan` (scans current package and sub-packages for components). It's the entry point of a Spring Boot application.

Q4. What is the difference between @Component, @Service, @Repository, and @Controller?

> All are specialisations of `@Component`. `@Service` marks business logic layer (semantic, no extra features). `@Repository` marks data access layer — additionally enables Spring's persistence exception translation (converts DB exceptions to Spring's DataAccessException hierarchy). `@Controller` marks web layer and integrates with MVC. `@RestController` = `@Controller` + `@ResponseBody`.

Q5. What is Spring Boot Actuator?

> Actuator provides production-ready features: health checks (`/actuator/health`), metrics (`/actuator/metrics`), environment info (`/actuator/env`), beans listing, thread dumps, log level changes. Endpoints can be exposed via HTTP or JMX. Essential for monitoring and operations.

Q6. What are Spring Boot Starters?

> Starters are dependency descriptors that include everything needed for a particular function. `spring-boot-starter-web` includes Spring MVC + Jackson + Tomcat. `spring-`

> `boot-starter-data-jpa` includes JPA + Hibernate + HikariCP. They manage transitive dependencies and version compatibility.

Q7. How does Spring Boot handle application properties?

> Properties are loaded from `application.properties` or `application.yml`. Profile-specific files `application-{profile}.yml` override the defaults. Properties can also come from environment variables, command-line args, and config server. Order of precedence (highest to lowest): command-line → env vars → profile config → application.yml.

Q8. What is the difference between @Value and @ConfigurationProperties?

> `@Value("${app.name}")` injects individual properties. `@ConfigurationProperties(prefix = "app")` binds a whole group of properties to a POJO — better for related properties, supports validation, type-safe, IDE-friendly. Always prefer `@ConfigurationProperties` for multiple related properties.

Q9. What is Spring Profile?

> Profiles allow defining beans and configuration for different environments. `@Profile("dev")` activates a bean only in the dev profile. Active profile set via `spring.profiles.active=dev` or `SPRING_PROFILES_ACTIVE=prod`. Used to have different DB configs, service URLs, or feature flags per environment.

Q10. What is dependency injection and what types does Spring support?

> DI is a design pattern where dependencies are provided rather than created by the class. Spring supports: Constructor injection (preferred — immutable, easy to test), Setter injection (optional dependencies), Field injection (convenient but avoid — hides dependencies, hard to test). Use constructor injection for mandatory, setter for optional.

Q11. What is the Bean scope in Spring?

> Singleton (default) — one instance per Spring container. Prototype — new instance each time requested. Request — one per HTTP request. Session — one per HTTP session. Application — one per ServletContext. WebSocket — one per WebSocket. Use `@Scope("prototype")` to change from singleton.

Q12. What is the difference between BeanFactory and ApplicationContext?

> Both are IoC containers. `BeanFactory` is lazy — beans created on first request. `ApplicationContext` is eager — beans created at startup. ApplicationContext adds: events, AOP, internationalization, resource loading, and environment abstraction. Always use ApplicationContext in Spring Boot.

Q13. How does @Transactional work in Spring?

> Spring wraps the method in a proxy. Before the method: begin transaction. After success: commit. On RuntimeException: rollback (checked exceptions don't rollback by default — use `rollbackFor`). Works with: propagation (REQUIRED, REQUIRES_NEW, NESTED), isolation levels, and read-only hint. Note: @Transactional only works when called from outside the bean (proxy limitation — self-invocation doesn't work).

Q14. What is AOP (Aspect-Oriented Programming) in Spring?

> AOP separates cross-cutting concerns (logging, security, transactions) from business logic. Key concepts: Aspect (the concern), Advice (what to do — @Before, @After, @Around), Pointcut (where to apply), JoinPoint (the method being intercepted). Spring uses proxy-based AOP.

Q15. Explain Spring Boot's embedded server.

> Spring Boot embeds a servlet container (Tomcat by default, or Jetty/Undertow) inside the JAR. No WAR deployment needed. The application runs as a regular Java application with `java -jar app.jar`. This enables: containerisation, microservices deployment, and simplified DevOps.

Q16. What is CommandLineRunner and ApplicationRunner?

> Both execute code after the Spring context is fully loaded. `CommandLineRunner.run(String... args)` receives raw command-line args. `ApplicationRunner.run(ApplicationArguments args)` receives parsed args. Use cases: DB seeding, cache warming, startup validation.

Q17. How does Spring Boot handle exceptions in REST APIs?

> Via `@RestControllerAdvice` + `@ExceptionHandler`. Define a global handler class annotated with `@RestControllerAdvice`. Each method annotated with `@ExceptionHandler(SomeException.class)` handles that exception type and returns a structured error response. Applied across all controllers.

Q18. What is ResponseEntity?

> A wrapper that gives full control over the HTTP response: status code, headers, and body. `ResponseEntity.ok(body)` returns 200. `ResponseEntity.created(location).body(body)` returns 201. `ResponseEntity.noContent().build()` returns 204. `ResponseEntity.status(HttpStatus.CONFLICT).body(error)` returns 409.

Q19. What is @RestController vs @Controller?

> `@Controller` is used with Spring MVC views — returns view names (Thymeleaf templates). `@RestController` = `@Controller` + `@ResponseBody` — every method's return value is serialised to JSON/XML and written directly to the HTTP response. Use `@RestController` for REST APIs.

Q20. What is content negotiation?

> The process where the client and server agree on the response format. Client sends `Accept: application/json` or `Accept: application/xml`. Server produces the format it supports (configured via `produces` in `@GetMapping`). Jackson handles JSON, JAXB handles XML. Spring selects the appropriate `HttpMessageConverter`.

Microservices Questions (Q21–Q50)

Q21. What are microservices and what are their advantages?

> Microservices is an architectural style breaking an app into small, independently deployable services communicating via APIs. Advantages: independent scaling, independent deployment, technology diversity, better fault isolation, easier to understand and maintain. Disadvantages: distributed systems complexity, network latency, eventual consistency challenges.

Q22. What is the difference between monolith and microservices?

> Monolith: single deployable unit, shared DB, simple to develop initially, complex to scale and maintain as it grows. Microservices: multiple deployable units, each with its own DB, complex operationally but scales better, each service independently deployable and maintainable by a small team.

Q23. What is Service Discovery and how does Eureka work?

> Service Discovery allows services to find each other dynamically without hardcoded IPs. Eureka works: each service registers with Eureka Server on startup (sends heartbeats every 30s). Services query Eureka to get instances of another service. Client-side load balancing (Ribbon/Spring Cloud LoadBalancer) picks an instance. Self-preservation mode prevents mass de-registration on network partition.

Q24. What is an API Gateway?

> Single entry point for all client requests to microservices. Responsibilities: routing requests to the correct service, JWT authentication/authorisation, rate limiting, load balancing, request/response transformation, SSL termination, caching, logging. Spring Cloud Gateway uses reactive Netty for non-blocking I/O.

Q25. What is a Circuit Breaker pattern?

> Prevents cascading failures when a service is down. States: CLOSED (normal), OPEN (service failing — fast fail without calling), HALF-OPEN (test recovery with limited calls). When the circuit is OPEN, a fallback response is returned immediately without waiting for a timeout. Resilience4j is the standard library in Spring Boot 3.

Q26. What is the Saga pattern?

> Manages distributed transactions across multiple services by decomposing into local transactions with compensating actions. Choreography: services react to events (decentralised, harder to track). Orchestration: a central orchestrator directs steps (easier to see the full flow, single point of failure). Used for e-commerce: order placement spanning inventory, payment, and shipping.

Q27. What is eventual consistency?

> In distributed systems, data across services may temporarily be inconsistent but will eventually reach consistency. Unlike ACID transactions (immediate consistency), microservices accept temporary inconsistency in exchange for availability and partition tolerance (CAP theorem).

Q28. How do you handle distributed transactions in microservices?

> Options: Saga pattern (most common), 2-Phase Commit (strong consistency but blocks), Outbox pattern (reliable event publishing), TCC (Try-Confirm-Cancel). The Outbox pattern: write business data AND event to the same DB transaction, a relay process publishes events to the message broker.

Q29. What is the Outbox pattern?

> Guarantees at-least-once delivery of events without distributed transactions. Steps: 1) In the same DB transaction, save entity AND event to an outbox table. 2) A poller reads uncommitted outbox events and publishes to Kafka/RabbitMQ. 3) Mark events as published. Eliminates the dual-write problem (DB + message broker).

Q30. What is the difference between synchronous and asynchronous communication?

> Synchronous (REST, gRPC): caller waits for response. Simple, easy to reason about, but creates temporal coupling — both services must be available simultaneously. Asynchronous (Kafka, RabbitMQ): caller publishes event and continues. Decoupled, more resilient, but harder to trace, eventual consistency required.

Q31. What is Spring Cloud?

> A suite of tools for building distributed systems with Spring Boot: Spring Cloud Netflix (Eureka, Ribbon), Spring Cloud Gateway, Spring Cloud Config, Spring Cloud OpenFeign, Spring Cloud Sleuth, Spring Cloud Circuit Breaker. It implements common microservices patterns on top of Spring Boot.

Q32. What is the Strangler Fig pattern?

> A migration strategy to gradually replace a monolith with microservices. The API Gateway routes some requests to microservices and others still to the monolith. Incrementally, more functionality is extracted until the monolith is "strangled" completely.

Q33. What is CQRS?

> Command Query Responsibility Segregation separates write operations (Commands) from read operations (Queries). Different models for reads and writes. Write model normalised for consistency. Read model denormalised for performance (potentially different DB). Scales reads and writes independently.

Q34. What is the Database per Service pattern?

> Each microservice owns its data store. No shared database between services. Ensures loose coupling — a service can change its DB without affecting others. Challenges: no ACID transactions across services, data duplication, eventual consistency required.

Q35. What is the difference between Kafka and RabbitMQ?

> Kafka: log-based, high throughput (millions/sec), message retention (replay possible), partition-based ordering, pull model. RabbitMQ: traditional queue, lower throughput but simpler, messages deleted after consumption, push model, supports complex routing. Use Kafka for event streaming and audit logs; RabbitMQ for task queues.

Q36. What is OpenFeign and how does it work?

> OpenFeign is a declarative HTTP client. You define an interface with annotations mapping to REST endpoints. At runtime, Spring generates a proxy implementation that makes actual HTTP calls. Integrates with Eureka (service name resolution), circuit breaker (fallback), and load balancing.

Q37. How do you implement security in microservices?

> JWT-based stateless authentication at the API Gateway. Gateway validates token, extracts user identity, and forwards in request headers. Downstream services trust the header (mTLS between internal services for added security). OAuth2/OIDC (Keycloak) for enterprise setups. Each service applies method-level authorisation (`@PreAuthorize`).

Q38. What is Spring Cloud Config?

> Centralised configuration server for all microservices. Stores config in Git, filesystem, or Vault. Services fetch their config on startup. `@RefreshScope` allows refreshing config without restart (triggered via `/actuator/refresh`). Spring Cloud Bus can broadcast refresh to all instances simultaneously.

Q39. How do you trace a request across multiple microservices?

> Distributed tracing assigns a TraceId to each request and SpanIds to each service call. All logs include the TraceId. Zipkin/Jaeger aggregate spans to show the full request path, timing, and errors. Spring Boot 3 uses Micrometer Tracing with OpenTelemetry.

Q40. What is Resilience4j?

> A fault-tolerance library for Java. Provides: CircuitBreaker (fail fast on upstream failure), Retry (retry failed calls with backoff), RateLimiter (limit calls per second), Bulkhead (limit concurrent calls), TimeLimiter (timeout for async calls). Each is composable and independently configurable.

Q41. What is the difference between RestTemplate and WebClient?

> RestTemplate is synchronous (blocks the thread while waiting). WebClient is reactive/async (non-blocking). WebClient is the modern choice (RestTemplate is in maintenance mode). Use WebClient for new projects, especially with high concurrency. Both support load balancing via `@LoadBalanced`.

Q42. What is the difference between load balancing client-side and server-side?

> Client-side: service fetches the list of instances from Eureka and picks one itself (Ribbon/Spring Cloud LoadBalancer). More flexible, no bottleneck. Server-side: a dedicated load balancer (Nginx, HAProxy, AWS ALB) distributes traffic — simpler for the client but adds infrastructure.

Q43. How would you implement rate limiting in Spring Cloud Gateway?

> Using the `RequestRateLimiter` filter with Redis: `filter name: RequestRateLimiter`, `redis-rate-limiter.replenishRate: 10` (10 requests/sec), `redis-rate-limiter.burstCapacity: 20`. Custom key resolver (by user, IP, or API key). Responds with 429 when limit exceeded.

Q44. What is service mesh and how does it relate to microservices?

> A service mesh (Istio, Linkerd) handles service-to-service communication infrastructure: mTLS encryption, traffic management (canary, retry, timeout), observability (metrics, tracing), policy enforcement. Implemented via sidecar proxies (Envoy). Moves networking concerns out of application code.

Q45. What is the difference between blue-green and canary deployments?

> Blue-Green: two identical environments. Switch traffic from blue (current) to green (new) instantly. Rollback = switch back. Canary: gradual traffic shift (5% → 20% → 50% → 100%) — test with real traffic before full rollout. Canary catches issues with a small blast radius.

Q46. How do you handle backward compatibility in APIs?

> Never break existing clients: add new optional fields (don't remove/rename existing ones), add new endpoints for major changes, use versioning (v1, v2), deprecate old versions with sunset headers. Consumer-Driven Contract Testing (Pact) ensures producer doesn't break consumers.

Q47. What is the difference between @Transactional in monolith vs microservices?

> In monolith: `@Transactional` uses ACID database transactions — all-or-nothing across multiple operations. In microservices: there's no single transaction across services. Each service has its own DB. Distributed consistency is achieved via Saga (eventual), 2PC (strong, rarely used), or Outbox pattern.

Q48. What is the Bulkhead pattern?

> Isolates resources (thread pools, semaphores) for different services to prevent one slow service from consuming all resources and failing others. Like a ship's bulkhead preventing one flooded compartment from sinking the whole ship. Resilience4j Bulkhead limits concurrent calls to a specific service.

Q49. How do you implement caching in Spring Boot microservices?

> `@Cacheable` on service methods with Redis or Caffeine backend. API Gateway can cache responses. Cache invalidation via `@CacheEvict`. Cache-aside pattern: check cache → if miss, fetch from DB → update cache. Distributed cache (Redis) for multiple service instances. Be careful with cache invalidation on updates.

Q50. What is Consumer-Driven Contract Testing?

> Testing approach where consumers define the contracts (expectations) they have on providers. Tools: Pact framework. Consumer writes and publishes Pact files. Provider runs tests against these Pacts to verify it satisfies all consumers. Prevents accidental API breaking changes in CI/CD pipelines.

Advanced/Scenario Questions (Q51–Q80)

Q51. How would you handle a microservice that is consistently slow?

> 1. Add timeout to the calling service (TimeLimiter). 2. Add circuit breaker to fast-fail after threshold. 3. Profile the slow service: check slow DB queries, N+1 queries, missing indexes, thread contention. 4. Add caching for frequent reads. 5. Scale horizontally. 6. Check if async processing is possible. 7. Add bulkhead to limit concurrent calls.

Q52. How would you migrate a monolith to microservices?

> Use the Strangler Fig pattern: 1) Identify bounded contexts in the monolith. 2) Add an API Gateway in front of the monolith. 3) Extract one service at a time (start with least coupled). 4) Route requests for extracted service through the gateway to the new microservice. 5) Repeat until monolith is empty. Keep the monolith running throughout migration.

Q53. How do you ensure data consistency across microservices?

> Use Saga pattern for distributed transactions. Use Outbox pattern for reliable event publishing. Use idempotent consumers (same message processed twice = same result). Use event sourcing for full audit trail. Accept eventual consistency where appropriate. Use compensating transactions for rollback.

Q54. How would you debug a latency issue in a microservices system?

> 1. Check distributed traces in Zipkin/Jaeger — identify which service is slow. 2. Check metrics in Grafana (CPU, memory, DB query time). 3. Check logs for slow operations. 4. Check for N+1 queries (`WARN org.hibernate.stat`). 5. Check external dependencies (third-party APIs, S3). 6. Check network between services. 7. Check GC logs for long pauses.

Q55. How would you implement an API that doesn't block the caller while processing?

> Return 202 Accepted with a task ID immediately. Process asynchronously (@Async, Kafka consumer). Provide a polling endpoint: `GET /tasks/{taskId}/status`. Or implement WebSocket/SSE for push notifications when done. Use the async/reactive pattern with WebFlux for non-blocking I/O throughout.

Q56. What is the difference between Eureka and Consul for service discovery?

> Eureka: AP (available and partition-tolerant), eventual consistency, simpler, built for AWS. Consul: CP (consistent and partition-tolerant), stronger consistency, also provides key-value store, health checking, service mesh. Consul is more complex but more feature-rich. Spring Cloud supports both.

Q57. How do you secure inter-service communication?

> mTLS (mutual TLS): both services authenticate each other with certificates. OAuth2 service-to-service: client credentials grant, services get tokens to call other services. API keys in headers (less secure). Network policies in Kubernetes restricting Pod-to-Pod traffic. Service mesh (Istio) handles mTLS transparently.

Q58. How would you implement feature flags in microservices?

> External configuration: store flag in Spring Cloud Config or Redis, toggle without redeployment. Tools: Unleash, LaunchDarkly, Flipt. Pattern: `if (featureFlag.isEnabled("new-pricing")) { newPricing() } else { oldPricing() }`. Enable gradual rollout (10% of users see new feature). A/B testing with persistent flag per user.

Q59. What is the difference between REST and gRPC?

> REST: text-based (JSON), human-readable, HTTP/1.1, widely supported, flexible. gRPC: binary (Protocol Buffers), strongly typed schema, HTTP/2 (multiplexing, streaming), higher

> performance, language-agnostic code generation. Use gRPC for high-performance internal service communication; REST for external/public APIs.

Q60. How do you handle secrets in a microservices production environment?

> Never hardcode secrets. Options: HashiCorp Vault (dynamic secrets, rotation), AWS Secrets Manager/Parameter Store, Kubernetes Secrets (with etcd encryption at rest), Spring Cloud Config with encrypted properties. Use the External Secrets Operator to sync from Vault to K8s Secrets. Rotate secrets regularly, audit access.

Q61. What is HATEOAS and should you implement it?

> Hypermedia as the Engine of Application State — responses include links to related actions: `_links: { self: /expenses/1, update: /expenses/1, delete: /expenses/1 }`. Level 3 of Richardson Maturity Model. Good for discoverability. Rarely implemented fully in practice — most APIs are Level 2 (resources + HTTP verbs). Use Spring HATEOAS if your API consumers need discoverability.

Q62. How would you implement pagination and sorting efficiently?

> Use Spring Data's `Pageable` and `Page`. Always paginate list endpoints (default size 20, max 100). Use cursor-based pagination for very large datasets (instead of offset — offset pagination slows with large pages). Index columns used for sorting. Return `totalElements` and `totalPages` for UI to render pagination controls.

Q63. Explain idempotency in REST APIs.

> An operation is idempotent if calling it multiple times has the same effect as calling once. GET, PUT, DELETE are idempotent. POST is not. Implement POST idempotency with idempotency keys (client sends UUID in header, server caches response by key, returns same response on duplicate). Critical for payment APIs.

Q64. What is CORS and how do you configure it in Spring Boot?

> Cross-Origin Resource Sharing — browsers block requests from one origin to another by default. Backend must include `Access-Control-Allow-Origin` header. In Spring Boot: `@CrossOrigin` on controller, or global config via `WebMvcConfigurer.addCorsMappings()`, or via Spring Security's `cors()` configuration. Always restrict allowed origins in production.

Q65. How do you implement API documentation?

> Springdoc OpenAPI (Swagger) — dependency auto-generates OpenAPI 3.0 spec from your controllers. Accessible at `/swagger-ui.html`. Annotate with `@Operation`, `@ApiResponse`, `@Tag` for detailed docs. Contract-first alternative: write OpenAPI spec first, generate server stubs with OpenAPI Generator.

Q66. What is the difference between @Mock, @MockBean, @Spy, and @SpyBean?

> `@Mock` (Mockito): creates mock outside Spring context. `@MockBean` (Spring Boot Test): creates mock AND registers it as a Spring bean, replacing real bean. `@Spy` (Mockito): wraps real object, stubs only specified methods. `@SpyBean`: spy on a Spring-managed bean. Use `@MockBean` in `@WebMvcTest`; `@Mock` in `@ExtendWith(MockitoExtension.class)`.

Q67. What is Testcontainers and why use it?

> Java library for spinning up Docker containers during tests. Use real PostgreSQL, Kafka, Redis in tests instead of H2 in-memory DB. Tests run against the same DB/broker as production — catches real issues. Containers start fresh per test class, ensuring test isolation. Essential for integration and repository tests.

Q68. How do you implement retry logic in microservices?

> Resilience4j `@Retry`: configure max attempts, wait duration, and exception types to retry. Exponential backoff prevents thundering herd. Jitter adds randomness to backoff. Only retry idempotent operations. Log retry attempts and alert on repeated failures. Circuit breaker sits above retry to prevent endless retrying of a dead service.

Q69. What is zero-downtime deployment and how do you achieve it?

> Deploy new version without service interruption. Techniques: rolling deployment (K8s default — gradually replaces old pods), blue-green (instant traffic switch), canary (gradual traffic shift). Requirements: backward-compatible API changes, database migrations must be backward-compatible (expand/contract), pods have `readinessProbe` so traffic only goes to ready instances.

Q70. What is event sourcing?

> Instead of storing current state, store all events that led to the state. Current state = replay all events. Provides: full audit trail, time travel (state at any point), event replay for new projections. More complex: CQRS usually combined, eventual consistency. Tools: Axon Framework, EventStore.

Q71. How do you implement health checks in Spring Boot microservices?

> Spring Boot Actuator's `/actuator/health` checks: database connectivity, message broker connectivity, disk space, custom health indicators. K8s `readinessProbe` → pod receives traffic only when ready. K8s `livenessProbe` → restart pod if hung. `/actuator/health/readiness` and `/actuator/health/liveness` for K8s.

Q72. What is the API Gateway vs Service Mesh distinction?

> API Gateway (North-South traffic): manages external client → cluster communication. Handles: auth, rate limiting, SSL termination. Service Mesh (East-West traffic): manages service → service communication within the cluster. Handles: mTLS, retries, circuit breaking, observability. They are complementary — use both in mature architectures.

Q73. How do you handle long-running processes in REST APIs?

> Return 202 Accepted with a task location header immediately. Process in a background thread (`@Async`) or async message consumer. Polling endpoint: `GET /tasks/{id}` returns status (PENDING/PROCESSING/COMPLETED/FAILED). WebSocket/SSE for push notification. Store progress in Redis or DB. This is the async request-reply pattern.

Q74. What is the Strangler Fig pattern?

> An incremental approach to migrating a legacy system. Named after the strangler fig tree that slowly envelops and replaces its host. Place an API Gateway (or reverse proxy) in front of the monolith. Incrementally route specific paths to new microservices. The monolith gradually shrinks as features are migrated. Zero big-bang rewrite risk.

Q75. How do you prevent N+1 query problems in Spring Data JPA?

> N+1 happens when fetching N entities triggers N additional queries to load associations. Solutions: `@EntityGraph` to eagerly fetch needed associations. `JOIN FETCH` in JPQL. `FetchType.LAZY` by default, `FetchType.EAGER` only when always needed. Use DTOs with `@Query` selecting only needed columns. Spring Data projections. Hibernate `@BatchSize`.

Q76. What is the difference between EAGER and LAZY loading in JPA?

> EAGER: association fetched immediately when entity is fetched (N+1 risk). LAZY: association fetched only when accessed (must be within transaction). Default: `@OneToOne` and `@ManyToOne` default EAGER; `@OneToMany` and `@ManyToMany` default LAZY. Always prefer LAZY and fetch eagerly only in specific queries where association is needed.

Q77. How do you implement audit logging in microservices?

> Spring Data's `@CreatedBy`, `@LastModifiedBy`, `@CreatedDate`, `@LastModifiedDate` with `AuditingEntityListener`. Enable with `@EnableJpaAuditing`. For distributed audit log: every significant operation publishes an audit event to Kafka → a dedicated audit service consumes and stores. Store: actor, action, resource, timestamp, before/after state.

Q78. What is the difference between domain events and integration events?

> Domain events: represent something that happened in the domain (ExpenseCreated). Internal to a bounded context. May trigger side effects within same service. Integration events: represent facts to share across service boundaries via message broker. Should be designed for backward compatibility.

Q79. How do you test a Feign client?

> WireMock: set up a mock HTTP server, configure Feign to point to it, verify correct requests are made. `@WireMockTest` annotation in Spring Boot tests. Test the fallback by simulating 5xx responses. Contract tests with Pact: verify Feign client matches provider's actual API.

Q80. What are the key metrics to monitor in a microservices system?

> RED metrics: Rate (requests/sec), Errors (error rate), Duration (latency p50/p99). USE metrics for infrastructure: Utilisation, Saturation, Errors. Business metrics: active users, orders created, revenue. Service-specific: DB connection pool usage, queue depth, circuit breaker state. Alert on: error rate spike, p99 latency increase, pod restarts, CPU/memory above threshold.

---

22. Quick Reference Cheat Sheet

HTTP Method → Status Code Map

```
GET    → 200 OK
POST   → 201 Created  (+Location header)
PUT    → 200 OK
PATCH  → 200 OK
DELETE → 204 No Content
```

Spring Annotations Summary

```java
```

```
// Controller layer
@RestController, @RequestMapping, @GetMapping, @PostMapping,
@PutMapping, @PatchMapping, @DeleteMapping
@PathVariable, @RequestParam, @RequestBody, @RequestHeader,
@ResponseStatus, @CrossOrigin

// Service layer
@Service, @Transactional, @Async, @Scheduled,
@CacheConfig, @Cacheable, @CacheEvict

// Repository layer
@Repository, @Query, @Modifying, @Param, @Lock

// Configuration
@Configuration, @Bean, @Value, @ConfigurationProperties,
@Profile, @Conditional, @Primary, @Qualifier

// Security
@EnableWebSecurity, @EnableMethodSecurity,
@PreAuthorize, @PostAuthorize, @Secured

// Spring Cloud
@EnableEurekaServer, @EnableEurekaClient,
@EnableFeignClients, @FeignClient,
@EnableConfigServer, @RefreshScope,
@CircuitBreaker, @Retry, @RateLimiter
```

Common Spring Boot Properties

```yaml
```

```yaml
# Server
server.port: 8080
server.servlet.context-path: /api

# JPA
spring.jpa.hibernate.ddl-auto: validate
spring.jpa.show-sql: false

# Actuator
management.endpoints.web.exposure.include: health,metrics,prometheus
management.endpoint.health.probes.enabled: true

# Logging
logging.level.com.example: DEBUG

# Spring Cloud
spring.application.name: expense-service
eureka.client.service-url.defaultZone: http://localhost:8761/eureka/
spring.cloud.gateway.routes[0].id: expense-service
spring.cloud.gateway.routes[0].uri: lb://expense-service
```

Microservices Port Convention

```
8080  API Gateway
8761  Eureka Server
8888  Config Server
8081  User Service
8082  Expense Service
8083  Budget Service
8084  Notification Service
9411  Zipkin
9090  Prometheus
3000  Grafana
5432  PostgreSQL
9092  Kafka
5672  RabbitMQ
6379  Redis
```

Debugging Checklist

```
Service not starting?
  ✓ Check port conflict (server.port)
```

```
  ✓ DB connection failing? Check datasource URL/credentials
  ✓ Bean creation failure? Check @Autowired dependencies
  ✓ Profile mismatch? Check spring.profiles.active

REST API returning 404?
  ✓ Check @RequestMapping URL matches
  ✓ Context path set? (server.servlet.context-path)
  ✓ Security blocking? Check SecurityConfig permitAll paths

REST API returning 500?
  ✓ Check application logs for full stack trace
  ✓ NullPointerException in service? Check null handling
  ✓ DB query failure? Enable show-sql

@Transactional not working?
  ✓ Self-invocation? (calling @Transactional from same class)
  ✓ Method must be public
  ✓ Rollback on checked exception? Add rollbackFor

Feign client failing?
  ✓ Target service registered in Eureka?
  ✓ Correct service name in @FeignClient(name=)?
  ✓ Fallback configured for timeout handling?
```

Technology Stack for Production

```
Application:     Spring Boot 3 + Java 21
ORM:             Spring Data JPA + Hibernate
DB:              PostgreSQL (primary) + Redis (cache)
Service Disc.:   Eureka Server
API Gateway:     Spring Cloud Gateway
Config:          Spring Cloud Config + Git
Messaging:       Apache Kafka
Circuit Breaker: Resilience4j
Tracing:         Micrometer + Zipkin/Jaeger
Monitoring:      Prometheus + Grafana + Actuator
Security:        Spring Security + JWT + OAuth2
Testing:         JUnit 5 + Mockito + Testcontainers
Documentation:   Springdoc OpenAPI (Swagger)
Deployment:      Docker + Kubernetes + Helm
CI/CD:           GitHub Actions / GitLab CI
```

Study tip: Build the expense tracker application from scratch — implement each section as you study it. The best interview preparation is having a real project where you have made actual architectural decisions and can explain why you chose each approach.